

PDLP: A Practical First-Order Method for Large-Scale Linear Programming

David Applegate* Mateo Díaz† Oliver Hinder‡ Haihao Lu§
 Miles Lubin¶ Brendan O’Donoghue|| Warren Schudy*

Abstract

We present PDLP, a practical first-order method for linear programming (LP) designed to solve large-scale LP problems. PDLP is based on the primal-dual hybrid gradient (PDHG) method applied to the minimax formulation of LP. PDLP incorporates several enhancements to PDHG, including diagonal preconditioning, presolving, adaptive step sizes, adaptive restarting, and feasibility polishing. Our algorithm is implemented in C++, available in Google’s open-source OR-Tools library, and supports multithreading.

To evaluate our method, we introduce a new collection of eleven large-scale LP problems with sizes ranging from 125 million to 6.3 billion nonzeros. PDLP solves eight of these instances to optimality gaps of 1% (with primal and dual feasibility errors of less than 10^{-8}) within six days on a single machine. We also compare PDLP with Gurobi barrier, primal simplex, and dual simplex implementations. These traditional methods are designed to solve linear programs to much tighter optimality gaps but struggle to solve these instances. Gurobi barrier solves only three instances, exceeding our 1TB RAM limit on the other eight. While primal and dual simplex are more memory-efficient than the barrier method, they are slower and solve only three instances within six days.

Compared with the conference version of this work (in: Advances in Neural Information Processing Systems 34 (NeurIPS 2021)), the key new contributions are: (i) feasibility polishing, a technique that quickly finds solutions that are approximately optimal but almost exactly feasible (without which only two of the eleven problems can be solved); (ii) a multithreaded C++ implementation available in Google OR-Tools; and (iii) a new collection of large-scale LP problems. Note that the conference version should be referred to for comparisons with SCS and ablation studies, which we do not repeat in this paper.

1 Introduction

Applications of linear programming (LP) are widespread and span many different fields [19]. Simplex [68] and interior-point methods [93] are the standard approaches to solving LP problems. They are widely used in practice and are known for their reliability and ability to solve instances to high numerical precision. However, these methods face scaling limitations when applied to extremely large problems. For interior point methods, the limiting factor is often the memory required by sparse matrix factorizations. While the simplex method is less likely to run out of memory because it factorizes a smaller matrix, it often requires prohibitively many iterations. Moreover, it is difficult to adapt these methods to exploit highly-parallel modern computing hardware such as multithreading, distributed computing, and graphics processing units (GPUs).

A parallel stream of research that has had relatively little impact on the practice of LP, first-order methods (methods that use function and gradient information) have undergone an intense period of study over the past twenty years [14, 26, 36, 52, 69, 78], resulting in incredible success in machine learning [57, 59]. The

*Google Research ({dapplegate, wschudy}@google.com);

†Johns Hopkins University (mateodd@jhu.edu);

‡University of Pittsburgh (ohinder@pitt.edu);

§MIT and Google Research (haihao@mit.edu);

¶Hudson River Trading (miles.lubin@gmail.com; work performed primarily while at Google Research)

||Google DeepMind (bodonoghue@deepmind.com);

benefit of these methods is that they have cheap iterations that can readily utilize highly parallel modern computing hardware and quickly obtain approximately optimal solutions, which are sufficient for most large-scale machine learning applications. Building on this literature, this paper enhances an existing first-order method, primal-dual hybrid gradient (PDHG), popularized by Chambolle and Pock [26] in the computer vision community, to develop a specialized first-order method for LP.

Despite their appealing computational properties, a longstanding barrier to adopting first-order methods for LP is their inability to reliably solve instances to high numerical precision in reasonable timeframes. For LP, it is especially critical to find feasible solutions, both primal and dual. Even small violations of primal constraints (for instance, flow conservation) are difficult for practitioners to interpret. Similarly, finding dual feasible solutions is just as essential because of LP’s use for providing bounds on the optimal objective value such as within branch-and-bound methods [92]. As a concrete example of the risks associated with an inaccurately solved LP problem, consider the ‘irish-electricity’ instance, an LP instance based on a unit commitment case study of the Irish electric grid [25, 70], which has been used in Mittelman’s LP benchmark [71]. On this instance, SCS 3.27 [79], a state-of-the-art first-order method with its default termination tolerance of 10^{-4} , terminates in 40 seconds. However, the objective value for the returned solution is 2.21×10^6 , which is off from the true optimal objective value¹ of 2.56×10^6 by more than 13%. On the other hand, SCS with a tighter termination tolerance of 10^{-8} does not terminate successfully within an hour. This issue—where moderate feasibility violations lead to such undesirable behavior, which is only detectable using solutions provided from a more accurate solver—justifies practitioners’ caution in adopting first-order methods for LP.

To tackle this reliability issue, we apply a series of enhancements to PDHG including: adaptive restarting, dynamic primal-dual step size selection, and diagonal preconditioning. We call this enhanced PDHG algorithm, PDLP (PDHG for LP). The impact of these enhancements was demonstrated in the conference version of this paper [9]. For example, for finding high accuracy solutions on a standard LP benchmark, NETLIB [42], our enhancements enabled a $75\times$ speed-up over PDHG without any enhancements and a $19\times$ speed-up [9, Table 24] over SCS [79], a first-order method for conic optimization.

Compared with the conference version of this paper [9], the new contributions of this paper include:

1. An implementation of PDLP in multithreaded C++ code which is now part of Google’s open-source OR-Tools package (<https://developers.google.com/optimization>). The conference version of our paper uses single-thread Julia code².
2. A new collection of large-scale LP instances with between 125 million and 6.3 billion nonzeros to benchmark our method and a set of experiments demonstrating favorable results compared to Gurobi, a state of the art commercial solver. In comparison, the conference paper focused on smaller standard benchmark instances: the largest problem tested had 90 million nonzeros.
3. An additional enhancement that allows our algorithm to rapidly find solutions that are feasible but only approximately optimal. Our computational results demonstrate this new heuristic is extremely effective for these large-scale problems.

Note that this paper does not subsume the conference version because we do not repeat its numerical experiments. These experiments were:

1. An ablation study that individually disabled each enhancement and also compared with reasonable alternatives.
2. Starting from a naive PDHG implementation, a demonstration of the incremental impact of each enhancement.
3. Comparisons with existing solvers (SCS and Gurobi) on instances from medium-size LP benchmarks and a large-scale LP example derived from the PageRank problem [21].

¹Found using Gurobi with crossover enabled.

²The prototype Julia code, FirstOrderLp.jl, is available at <https://github.com/google-research/FirstOrderLp.jl>.

Given the C++ code is the exclusive focus of this paper, we decided neither to repeat results verbatim from the conference paper nor rerun these experiments using the C++ code as the conclusions would largely be the same. For reference [66, Table 8] shows that the single threaded C++ code achieves roughly a 1.5× speed-up over the Julia code.

Outline Section 2 introduces the PDHG algorithm and the notation we use throughout the paper. Section 3 explains the practical improvements we have added to the PDHG algorithm. Section 4 introduces our new feasibility polishing algorithm. Section 5 describes the C++ implementation. Section 6 describes the collection of large-scale instances. Section 7 presents our numerical results.

1.1 Related literature

Classic methods for linear programming. The simplex method was the first successful algorithm for LP, having been developed by Dantzig and others in the late 1940s, with improvements over the subsequent decades as the economic importance of LP grew [68]. The simplex method begins at a vertex and pivots between vertices, monotonically improving the objective function with each step. Interior-point methods [93] were a major breakthrough in the 1980s, coupling much stronger theoretical guarantees with practically successful implementations. In contrast to the simplex method, interior-point methods (IPMs) traverse the interior of the feasible region, progressing toward optimality by following a so-called central path [84]. Commercial and open-source solvers such as Gurobi, CPLEX, COPT, and HiGHS leverage both of these methods, often in combination, to provide reliable and accurate solutions to LP problems.

There have also been several papers that modify interior point methods so that instead of using a factorization to solve the linear systems that arise at each iteration they use preconditioned iterative methods [33, 40, 47, 48, 98]. This reduces the memory overhead of these methods and the cost of each inner iteration performed by the iterative method (e.g., conjugate gradient) is roughly the cost of a matrix-vector multiplication (assuming the preconditioner is cheap to implement). For example, Fountoulakis et al. [40] demonstrate excellent performance of these methods on large-scale compressed sensing problems.

PDHG. The primal-dual hybrid gradient (PDHG) method, originally proposed in [39, 99], has been extended and analyzed by numerous researchers [5, 26, 28, 30, 39, 51, 83]. PDHG is a form of operator splitting [13, 88], and it is closely related to the alternating direction method of multipliers (ADMM) and Douglas-Rachford splitting (DRS) [20, 38, 77], all of which can be interpreted as preconditioned instances of the proximal point method (PPM) [38, 63, 80, 86]. However, unlike ADMM, DRS, or PPM, when applied to LP problems, PDHG is matrix-free, relying solely on matrix-vector multiplications.

PDLP. The conference version of this work [9] introduced PDLP, a restarted PDHG with several practical enhancements specialized for solving LP problems. Experiments were based on a prototype Julia implementation. This paper includes an additional enhancement, and the experiments discussed in this paper are based on a C++ implementation of PDLP, which has been open-sourced through Google OR-Tools since 2022 [81]. More recently, a GPU implementation, dubbed cuPDLP, demonstrated strong numerical performance comparable to commercial LP solvers, which garnered attention from both the academic community and the solver industry [66, 67]. Variants of (cu)PDLP have been incorporated into both commercial and open-source optimization solvers, including COPT [43], FICO Xpress [95], HiGHS [56], and NVIDIA cuOpt [18].

The success of PDLP has spurred theoretical analysis. Lu and Yang [62] proved that PDHG exhibits a linear convergence rate when applied to LP. Applegate et al. [10] showed that a faster linear convergence rate can be achieved through restarts, with the rate matching the worst-case lower bound. Applegate et al. [11] demonstrated that for infeasible or unbounded LP instances, PDHG iterates diverge toward a certificate of infeasibility or unboundedness, allowing for infeasibility detection without additional computation. Lu and Yang [65] established that PDHG displays a two-stage behavior when applied to LP: in the first stage, it identifies the optimal active variables in finitely many iterations; in the second stage, it solves a homogeneous linear inequality system at an improved linear rate. This two-stage behavior helps explain the slow convergence observed when an LP instance is near-degenerate. Hinder [54] shows an iteration complexity

bound for restarted PDHG applied to totally unimodular LP of $O(Hm^{2.5}\sqrt{\text{nnz}(A)}\ln(Hn/\epsilon))$ where H is the largest absolute value of the right-hand side and objective vector, n is the number of columns, m is the number of rows, $\text{nnz}(A)$ the number of nonzeros in the constraint matrix, and ϵ is the distance to optimality measured in Euclidean norm. Unlike other results, this bound does not involve any condition numbers. Xiong and Freund [94] introduced the concept of level-set geometry, showing that PDHG converges more quickly for LP instances with regular, non-flat level sets around the optimal solution. Recently, additional numerical improvements have been proposed to further accelerate the algorithm. In particular, Xiong and Freund [94] introduced a central-path-based preconditioner for PDHG that significantly improves convergence. Further, Lu and Yang [64] incorporated the Halpern iteration [50] into PDLP, achieving accelerated theoretical guarantees and improved practical performance.

Other first-order methods for LP. Lan, Lu, and Monteiro [60] and Renegar [85] developed first-order methods (FOMs) for LP as a particular case of semidefinite programming with sublinear convergence rates. The FOM-based solvers apply to more general problem classes like cone or quadratic programming. In contrast, some of the enhancements that constitute PDLP are specialized, either in theory or practice, for LP (namely restarts [10] and presolving). A number of authors [37, 44, 72, 91, 96] have proposed linearly convergent FOMs for LP; however, to our knowledge, none have been the subject of a comprehensive computational study. FOM-based software packages that can be used to solve LP include the following:

- SCS [74, 75] is an ADMM-based solver for convex quadratic cone programs implemented in C. SCS applies ADMM [20] to the homogeneous self-dual embedding of the problem [97], which yields infeasibility certificates when appropriate. One of the key computational challenges of ADMM-based methods is solving a linear system at every iteration. SCS provides two options to tackle this subroutine: a direct solver, which computes and reuses the factorization of the matrix, and an indirect solver, which uses conjugate gradient methods to iteratively solve these linear systems, offering scalability trade-offs.
- OSQP [89] is another ADMM-based solver for convex quadratic programming implemented in C. Compared to SCS, a major difference is that OSQP applies ADMM to the original problem rather than the homogeneous self-dual embedding. It also has direct and indirect options for solving the linear systems at each iteration. A related solver, COSMO [41], uses the same algorithm for convex quadratic cone programs and is implemented in Julia.
- ECLIPSE [12] is a distributed LP solver that employs accelerated gradient descent to solve a smoothed dual formulation of LPs. It is specifically designed to handle large-scale LPs with particular decomposition structures, such as those encountered in web applications.
- ABIP and ABIP+ [61] are interior-point method (IPM) solvers for cone programming. ABIP uses a few steps of ADMM to approximately solve the sub-problems that arise when applying a path-following barrier algorithm to the homogeneous self-dual embedding of the problem. Although IPMs are technically second-order methods, the use of ADMM in the inner loop allows the solver to scale to larger problems than traditional IPMs can handle. ABIP+ [34] adapts many of the improvements from PDLP to ABIP to improve its performance.
- TFOCS [15] is a MATLAB solver designed for solving a variety of convex cone problems that arise in areas such as signal processing, machine learning, and statistics, including applications like the Dantzig selector, basis pursuit denoising, and image recovery. In principle, TFOCS can also be applied to LP. It utilizes a variant of Nesterov’s accelerated method [73] to solve a smoothed version of the primal-dual conic problem.
- ALGENCAN [7] is a widely established software package for nonlinear programming that is based on the augmented Lagrangian method. The augmented Lagrangian subproblems are solved by the inner solver GENCAN [17], which utilizes active-set strategies and spectral projected gradients. Crucially, ALGENCAN is designed to operate in a fully matrix-free manner, i.e., only using gradient evaluations and Hessian-vector products.

2 Preliminaries

In this section, we introduce the notation we use throughout the paper, summarize the LP formulations we solve, and introduce the baseline PDHG algorithm.

Notation. Let $\mathbb{R}$ denote the set of real numbers, $\mathbb{R}^+$ the set of nonnegative real numbers, and $\mathbb{R}^-$ the set of nonpositive real numbers. Let $\mathbb{N}$ denote the set of natural numbers (starting from one). We endow $\mathbb{R}^n$ with the standard dot product $\langle x, y \rangle = x^\top y$ and its induced norm $\|\cdot\|_2$. We will also use $\|\cdot\|_p$ to denote the ℓ_p norm for a vector. The ℓ_∞ -norm, $\|v\|_\infty$, computes the maximum absolute value of the vector v . For a vector $v \in \mathbb{R}^n$, we use v^+ and v^- for their positive and negative parts, i.e., $v_i^+ = \max\{0, v_i\}$ and $v_i^- = -\min\{0, v_i\}$. The symbol $\mathbf{1}$ denotes the vector of all ones. Given a matrix $A \in \mathbb{R}^{m \times n}$, the symbols $A_{i,\cdot}$ and $A_{\cdot,j}$ correspond to the i th row and j th column of A , respectively. We use $\|\cdot\|_2$ to denote the spectral norm for a matrix, i.e., its top singular value. We use $z^k = (x^k, y^k)$ to denote the k th iterate of PDHG and $z^{n,t} = (x^{n,t}, y^{n,t})$ to denote iterates of PDL where n is the outer iteration counter and t is the inner iteration counter.

Let $\mathcal{X} \subseteq \mathbb{R}^n$ be a closed convex set. The orthogonal projection onto $\mathcal{X}$ is given by

$$\mathbf{proj}_{\mathcal{X}}(x) = \operatorname{argmin}_{w \in \mathcal{X}} \|x - w\|_2.$$

Define the mapping $p: \mathbb{R}^n \times (\mathbb{R} \cup \{-\infty\})^n \times (\mathbb{R} \cup \{\infty\})^n \rightarrow \mathbb{R} \cup \{\infty\}$ given by

$$p(y; \ell, u) := u^\top y^+ - \ell^\top y^-.$$

The function $p(\cdot; \ell, u)$ is piecewise linear and convex for any two vectors $\ell \in (\mathbb{R} \cup \{-\infty\})^n$ and $u \in (\mathbb{R} \cup \{\infty\})^n$ satisfying $\ell \leq u$. With a slight abuse of notation, we will also use p to denote an analogous function taking m -dimensional inputs as implied by the context.

Linear Programming. We solve primal-dual LP problems of the form [90]:

$$\begin{array}{ll} \underset{x \in \mathbb{R}^n}{\text{minimize}} & c^\top x \\ \text{subject to:} & \ell_c \leq Ax \leq u_c \\ & \ell_v \leq x \leq u_v, \end{array} \quad \begin{array}{ll} \underset{y \in \mathbb{R}^m, r \in \mathbb{R}^n}{\text{maximize}} & -p(-y; \ell_c, u_c) - p(-r; \ell_v, u_v) \\ \text{subject to:} & c - A^\top y = r \\ & y \in \mathcal{Y} \\ & r \in \mathcal{R}. \end{array} \quad (1)$$

where $A \in \mathbb{R}^{m \times n}$, $c \in \mathbb{R}^n$, $\ell_c \in (\mathbb{R} \cup \{-\infty\})^m$, $u_c \in (\mathbb{R} \cup \{\infty\})^m$, $\ell_v \in (\mathbb{R} \cup \{-\infty\})^n$, $u_v \in (\mathbb{R} \cup \{\infty\})^n$, and the sets $\mathcal{Y} \subseteq \mathbb{R}^m$ and $\mathcal{R} \subseteq \mathbb{R}^n$ are Cartesian products with their i th components given by

$$\mathcal{Y}_i := \begin{cases} \{0\} & (\ell_c)_i = -\infty, (u_c)_i = \infty, \\ \mathbb{R}^- & (\ell_c)_i = -\infty, (u_c)_i \in \mathbb{R}, \\ \mathbb{R}^+ & (\ell_c)_i \in \mathbb{R}, (u_c)_i = \infty, \\ \mathbb{R} & \text{otherwise;} \end{cases} \quad \text{and} \quad \mathcal{R}_i := \begin{cases} \{0\} & (\ell_v)_i = -\infty, (u_v)_i = \infty, \\ \mathbb{R}^- & (\ell_v)_i = -\infty, (u_v)_i \in \mathbb{R}, \\ \mathbb{R}^+ & (\ell_v)_i \in \mathbb{R}, (u_v)_i = \infty, \\ \mathbb{R} & \text{otherwise;} \end{cases}$$

which are the values of y and r with a finite dual objective value. We highlight that the formulation we consider (1) is slightly different from the formulation in the conference version of this paper [9], which only considers a lower bound $\ell_c \leq Ax$ and has additional equality constraints $Gx = q$.

This pair of primal-dual problems is equivalent to the minimax problem [90]:

$$\min_{x \in \mathcal{X}} \max_{y \in \mathcal{Y}} \mathcal{L}(x, y) := c^\top x - y^\top Ax - p(y; -u_c, -\ell_c) \quad (2)$$

with $\mathcal{X} := \{x \in \mathbb{R}^n : \ell_v \leq x \leq u_v\}$. Notice that the variable r in (1) does not appear explicitly in the minimax formulation. This absence leaves an algorithmic choice for recovering the reduced costs, r , which impacts when the termination criteria is satisfied but does not affect the iterates themselves. A natural choice is to set $r = \mathbf{proj}_{\mathcal{R}}(c - A^\top y)$; we discuss a more advanced choice in Appendix A.

PDHG. When specialized to (2), the PDHG algorithm takes the form:

$$x^{k+1} = \underset{x}{\mathbf{proj}}(x^k - \tau(c - A^\top y^k)) \quad (3a)$$

$$y^{k+1} = y^k - \sigma A(2x^{k+1} - x^k) - \underset{[-u_c, -\ell_c]}{\mathbf{proj}}(\sigma^{-1}y^k - A(2x^{k+1} - x^k)) \quad (3b)$$

where $\tau, \sigma > 0$ are primal and dual step sizes, respectively. For completeness we provide the derivation of Equation (3) from PDHG in Appendix B. A quick case-by-case argument shows $y^{k+1} \in \mathcal{Y}$. For instance, suppose the i -th coordinate bounds satisfy $(\ell_c)_i = -\infty$ and $(u_c)_i \in \mathbb{R}$, and let $\hat{y}_i^k = y_i^k - \sigma A(2x^{k+1} - x^k)$. Then, we have

$$y_i^{k+1} = \begin{cases} 0 & \text{if } \hat{y}_i^k \geq -\sigma(u_c)_i, \\ \hat{y}_i^k + \sigma(u_c)_i & \text{otherwise,} \end{cases}$$

and, thus, in any case $y_i^{k+1} \in \mathbb{R}^- = \mathcal{Y}_i$, as we wanted. The other cases follow analogously.

PDHG is known to converge to an optimal solution when $\tau\sigma\|A\|_2^2 < 1$ [27, 30]. We reparameterize the step sizes by

$$\tau = \eta/\omega \quad \text{and} \quad \sigma = \omega\eta \quad \text{with } \eta \in (0, \infty) \quad \text{and} \quad \omega \in (0, \infty). \quad (4)$$

We call $\omega \in (0, \infty)$ the *primal weight*, and $\eta \in (0, \infty)$ the *step size*. Under this reparameterization, PDHG converges for all $\eta < 1/\|A\|_2$. This parameterization allows us to control the scaling between the primal and dual iterates with a single parameter ω . From now on and for the rest of the paper, we will use z as a placeholder for the concatenation of primal and dual vectors (x, y) . We use the term *primal weight* to describe ω because it is the weight on the primal variables in the following norm:

$$\|z\|_\omega := \sqrt{\omega\|x\|_2^2 + \frac{\|y\|_2^2}{\omega}}.$$

This norm plays a role in the theory for PDHG [27] and later algorithmic discussions.

3 Practical algorithmic improvements

In this section, we briefly describe algorithmic enhancements for PDHG that were introduced in the conference version of this paper [9]. We omit a discussion of presolve because we do not use it for the numerical experiments in this paper; see also the mention of presolve in Section 5. A separate section (Section 4) is reserved to introduce feasibility polishing, which is new to this work.

Note that while our enhancements are inspired by theory, our focus is on practical performance. The algorithm as a whole does not have convergence guarantees. Guarantees for certain individual enhancements are noted at the end of the section.

Algorithm 1 presents pseudo-code for PDLP. We modify the step sizes, add restarts, dynamically update the primal weights, and use multithreading. Before running Algorithm 1 we apply diagonal preconditioning.

Periodic checks of restarts and termination criteria There are some minor differences between the pseudo-code and the actual code. In particular, we only evaluate the restart or termination criteria (Line 10) every 64 iterations³. This reduces the associated overhead of these evaluations with minimal impact on the total number of iterations. We also check the termination criteria before beginning the algorithm and if we detect a numerical error.

³Condition (iii) of the restart criteria is checked every iteration.

Algorithm 1: PDLP (after preconditioning and presolve)

```

1 Input: An initial solution  $z^{0,0}$ ;
2 Initialize outer loop counter  $n \leftarrow 0$ , total iterations  $k \leftarrow 0$ , step size  $\hat{\eta}^{0,0} \leftarrow 1/\|A\|_\infty$ , primal weight
    $\omega^0 \leftarrow \text{InitializePrimalWeight}(c, \ell_c, u_c)$ ;
3 repeat
4    $t \leftarrow 0$ ;
5   repeat
6      $z^{n,t+1}, \eta^{n,t+1}, \hat{\eta}^{n,t+1} \leftarrow \text{AdaptiveStepOfPDHG}(z^{n,t}, \omega^n, \hat{\eta}^{n,t}, k)$  ;
7      $\bar{z}^{n,t+1} \leftarrow \frac{1}{\sum_{i=1}^{t+1} \eta^{n,i}} \sum_{i=1}^{t+1} \eta^{n,i} z^{n,i}$  ;
8      $z_c^{n,t+1} \leftarrow \text{GetRestartCandidate}(z^{n,t+1}, \bar{z}^{n,t+1}, z^{n,0})$  ;
9      $t \leftarrow t + 1, k \leftarrow k + 1$  ;
10  until restart or termination criteria holds;
11  restart the outer loop.  $z^{n+1,0} \leftarrow z_c^{n,t}, \hat{\eta}^{n+1,0} \leftarrow \hat{\eta}^{n,t}, n \leftarrow n + 1$  ;
12   $\omega^n \leftarrow \text{PrimalWeightUpdate}(z^{n,0}, z^{n-1,0}, \omega^{n-1})$  ;
13 until termination criteria holds;
14 Output:  $z^{n,0}$ .

```

Algorithm 2: One step of PDHG using our step size heuristic

```

1 Function AdaptiveStepOfPDHG ( $z^{n,t}, \omega^n, \hat{\eta}^{n,t}, k$ ):
2    $(x, y) \leftarrow z^{n,t}, \eta \leftarrow \hat{\eta}^{n,t}$  ;
3   for  $i = 1, \dots, \infty$  do
4      $x' \leftarrow \text{proj}_{\mathcal{X}}(x - \frac{\eta}{\omega^n}(c - A^\top y))$  ;
5      $y' \leftarrow y - \eta \omega^n A(2x' - x) - \eta \omega^n \text{proj}_{[-u_c, -\ell_c]}((\eta \omega^n)^{-1} y - A(2x' - x))$ ;
6      $\bar{\eta} \leftarrow \begin{cases} \frac{\|(x' - x, y' - y)\|_{\omega^n}^2}{2(y' - y)^\top A(x' - x)} & 2(y' - y)^\top A(x' - x) > 0 \\ \infty & \text{otherwise} \end{cases}$  ;
7      $\eta' \leftarrow \min((1 - (k + 1)^{-0.3})\bar{\eta}, (1 + (k + 1)^{-0.6})\eta)$  ;
8     if  $\eta \leq \bar{\eta}$  then
9       return  $(x', y'), \eta, \eta'$ 
10    end
11     $\eta \leftarrow \eta'$  ;
12 end

```

Step size choice. The convergence analysis [27, Equation (15)] of standard PDHG (3) relies on a small constant step size

$$\eta \leq \frac{\|z^{k+1} - z^k\|_\omega^2}{2(y^{k+1} - y^k)^\top A(x^{k+1} - x^k)} \quad (5)$$

where $z^k = (x^k, y^k)$. Classically one would ensure (5) by picking $\eta = \frac{1}{\|A\|_2}$. This is overly pessimistic and requires estimation of $\|A\|_2$. Instead our **AdaptiveStepOfPDHG** adjusts η dynamically to ensure that (5) is satisfied. If (5) isn't satisfied, we abort the step; i.e., we reduce η , and try again. If (5) is satisfied we accept the step. This is described in Algorithm 2. Note that in Algorithm 2 $\bar{\eta} \geq \frac{1}{\|A\|_2}$ holds always, and from this one can show the step size η satisfies $\eta \geq \frac{1-o(1)}{\|A\|_2}$ as k tends to infinity.

This step size is compared against other standard step size schemes in Appendix C.1 of Applegate et al. [9].

Adaptive restarts. In PDLP, we adaptively restart the PDHG algorithm at each outer iteration. To decide when to restart at the n th outer iteration we use the normalized duality gap at z , which for any radius $r \in (0, \infty)$, is defined by

$$\rho_r^n(z) := \frac{1}{r} \max_{(\hat{x}, \hat{y}) \in \{\hat{z} \in Z : \|\hat{z} - z\|_{\omega^n} \leq r\}} \{\mathcal{L}(x, \hat{y}) - \mathcal{L}(\hat{x}, y)\}$$

where $Z = \mathcal{X} \times \mathcal{Y}$ and for completeness define $\rho_0(z) := \limsup_{r \rightarrow 0^+} \rho_r^n(z)$. This measure was introduced by Applegate et al. [10]. Unlike the standard duality gap, $\max_{(\hat{x}, \hat{y}) \in Z} \{\mathcal{L}(x, \hat{y}) - \mathcal{L}(\hat{x}, y)\}$, the normalized duality gap is always a finite quantity. Furthermore, for any value of r and ω^n , the normalized duality gap $\rho_r^n(z)$ is 0 if and only if the solution z is an optimal solution to (2) [10]; thus, it provides a metric for measuring progress towards the optimal solution. We compute the normalized duality gap based on the linear time implementation given by Applegate et al. [10, Section 3.2]. For brevity, define $\mu_n(z, z_{\text{ref}})$ as the normalized duality gap at z with radius $\|z - z_{\text{ref}}\|_{\omega^n}$, i.e.,

$$\mu_n(z, z_{\text{ref}}) := \rho_{\|z - z_{\text{ref}}\|_{\omega^n}}^n(z),$$

where z_{ref} is a user-chosen reference point.

Choosing the restart candidate. To choose the restart candidate $z_c^{n,t+1}$ we call

$$\text{GetRestartCandidate}(z^{n,t+1}, \bar{z}^{n,t+1}, z^{n,0}) := \begin{cases} z^{n,t+1} & \mu_n(z^{n,t+1}, z^{n,0}) < \mu_n(\bar{z}^{n,t+1}, z^{n,0}) \\ \bar{z}^{n,t+1} & \text{otherwise} \end{cases}.$$

This choice is justified in Remark 5 of [10].

Restart criteria. We define three parameters: $\beta_{\text{necessary}} \in (0, 1)$, $\beta_{\text{sufficient}} \in (0, \beta_{\text{necessary}})$ and $\beta_{\text{artificial}} \in (0, 1)$. In PDLP we use $\beta_{\text{necessary}} = 0.9$, $\beta_{\text{sufficient}} = 0.1$, and $\beta_{\text{artificial}} = 0.5$. The algorithm restarts if one of three conditions holds:

- (i) **(Sufficient decay in normalized duality gap)** $\mu_n(z_c^{n,t+1}, z^{n,0}) \leq \beta_{\text{sufficient}} \mu_n(z^{n,0}, z^{n-1,0})$,
- (ii) **(Necessary decay + no local progress in normalized duality gap)**

$$\mu_n(z_c^{n,t+1}, z^{n,0}) \leq \beta_{\text{necessary}} \mu_n(z^{n,0}, z^{n-1,0}) \quad \text{and} \quad \mu_n(z_c^{n,t+1}, z^{n,0}) > \mu_n(z_c^{n,t}, z^{n,0}),$$

- (iii) **(Long inner loop)** $t \geq \beta_{\text{artificial}} k$.

The motivation for (i) is presented in [10]; it guarantees the linear convergence of restarted PDHG on LP problems. The second condition in (ii) is inspired by adaptive restart schemes for accelerated gradient descent where restarts are triggered if the function value increases [78]. The first inequality in (ii) provides a safeguard for the second one, preventing the algorithm from restarting every inner iteration or never restarting. The motivation for (iii) relates to the primal weight updates. In particular, primal weight updates only occur after a restart (see Line 11 of Algorithm 1), and condition (iii) ensures that the primal weight will be updated

infinitely often. This prevents a bad choice of primal weight in earlier iterations from causing progress to stall for a long time.

The practical improvement from our restart scheme is demonstrated in Appendix C.2 of Applegate et al. [9].

Primal weight updates. Algorithm 3 aims to choose the primal weight ω^n such that distance to optimality from primal and dual iterates is the same, i.e., $\|(x^{n,t} - x^*, \mathbf{0})\|_{\omega^n} \approx \|(\mathbf{0}, y^{n,t} - y^*)\|_{\omega^n}$. By definition of $\|\cdot\|_{\omega}$,

$$\|(x^{n,t} - x^*, \mathbf{0})\|_{\omega^n} = \sqrt{\omega^n} \|x^{n,t} - x^*\|_2, \quad \|(\mathbf{0}, y^{n,t} - y^*)\|_{\omega^n} = \frac{1}{\sqrt{\omega^n}} \|y^{n,t} - y^*\|_2.$$

Setting these two terms equal yields $\omega^n = \frac{\|y^{n,t} - y^*\|_2}{\|x^{n,t} - x^*\|_2}$. Evidently, the quantity $\frac{\|y^{n,t} - y^*\|_2}{\|x^{n,t} - x^*\|_2}$ is unknown beforehand, but we attempt to estimate it using Δ_y^n / Δ_x^n . However, the quantity Δ_y^n / Δ_x^n can change wildly from one restart to another, causing ω^n to oscillate. To dampen variations in ω^n , we first move to a log-scale, in which the primal weight is symmetric, i.e., $\log(1/\omega^n) = -\log(\omega^n)$, and perform an exponential smoothing with parameter $\theta \in [0, 1]$. In PDLF, we use $\theta = 0.5$. The parameter $\epsilon_{\text{zero}} = 10^{-10}$ is a small nonzero tolerance that, in principle, could be adjusted. It helps prevent numerical issues or other performance issues occurring from very huge or very small movement of the iterates resulting in a very dramatic change in the primal weights.

Algorithm 3: Primal weight update

```

1 Function PrimalWeightUpdate( $z^{n,0}, z^{n-1,0}, \omega^{n-1}$ ):
2    $\Delta_x^n = \|x^{n,0} - x^{n-1,0}\|_2, \quad \Delta_y^n = \|y^{n,0} - y^{n-1,0}\|_2$ ;
3   if  $\Delta_x^n \in (\epsilon_{\text{zero}}, \epsilon_{\text{zero}}^{-1})$  and  $\Delta_y^n \in (\epsilon_{\text{zero}}, \epsilon_{\text{zero}}^{-1})$  then
4     return  $\exp\left(\theta \log\left(\frac{\Delta_y^n}{\Delta_x^n}\right) + (1 - \theta) \log(\omega^{n-1})\right)$ 
5   else
6     return  $\omega^{n-1}$ ;
7   end

```

Initialization of the primal weights. Define the map $\text{Combine}(\ell_c, u_c)$ which outputs a vector in $\mathbb{R}^m$ whose i th coordinate is given by $\max\{0, (\ell_c)_i^*, (u_c)_i^*\}$ where for any $x \in \mathbb{R} \cup \{\pm\infty\}$, the operation $(x)^*$ is equal to $|x|$ if it is finite or equal to zero otherwise. The primal weight is initialized using

$$\text{InitializePrimalWeight}(c, \ell_c, u_c) := \begin{cases} \frac{\|c\|_2}{\|\text{Combine}(\ell_c, u_c)\|_2} & \|c\|_2 > 0.0 \text{ and } \|\text{Combine}(\ell_c, u_c)\|_2 > 0.0 \\ 1 & \text{otherwise.} \end{cases}$$

There are several important differences between our primal weight heuristic and proposals from Goldstein et al. [45, 46]. For example, Goldstein et al. make relatively small changes to the primal weights at each iteration, attempting to balance the primal and dual residual. These changes have to be diminishingly small because, in our experience, PDHG may be unstable if they are too big. In contrast, in our method, the primal weight is only updated during restarts, which in practice allows for much larger changes without instability issues. Moreover, our scheme tries to balance the weighted distance traveled by the primal and dual iterates rather than the residuals.

The practical improvement from our primal weight scheme is demonstrated in Appendix C.3 of Applegate et al. [9].

Diagonal Preconditioning. Preconditioning is a popular heuristic in optimization for improving the convergence of first-order methods. To avoid factorizations, we only consider diagonal preconditioners. Our goal is to rescale the constraint matrix A to $\tilde{A} = D_1 A D_2$ with positive diagonal matrices D_1 and D_2 , so that the resulting matrix $\tilde{A}$ is “well balanced”. Such preconditioning creates a new LP instance that replaces

$A, c, (\ell_c, u_c)$, and (ℓ_v, u_v) in (1) with $\tilde{A}, \tilde{x} = D_2^{-1}x$, $\tilde{c} = D_2c$, $(\tilde{\ell}_c, \tilde{u}_c) = D_1(\ell_c, u_c)$, and $(\tilde{\ell}_v, \tilde{u}_v) = D_2^{-1}(\ell_v, u_v)$. Common choices for D_1 and D_2 include:

- **Pock-Chambolle [82]:** Pock and Chambolle proposed a family of diagonal preconditioners⁴ for PDHG parameterized by α , where the diagonal matrices are defined by $(D_1)_{jj} = 1/\sqrt{\|A_{j,\cdot}\|_{2-\alpha}}$ for $j = 1, \dots, m$ and $(D_2)_{ii} = 1/\sqrt{\|A_{\cdot,i}\|_\alpha}$ for $i = 1, \dots, n$. We use $\alpha = 1$ in PDLP. This is the baseline diagonal preconditioner in the PDHG literature.
- **Ruiz [87]:** In an iteration of Ruiz scaling, the diagonal matrices are defined as $(D_1)_{jj} = 1/\sqrt{\|A_{j,\cdot}\|_\infty}$ for $j = 1, \dots, m$ and $(D_2)_{ii} = 1/\sqrt{\|A_{\cdot,i}\|_\infty}$ for $i = 1, \dots, n$. Ruiz [87] shows that if this rescaling is applied iteratively, the infinity norm of each row and each column converge to 1.

For the default PDLP settings, we apply a combination of Ruiz rescaling [87] and the preconditioning technique proposed by Pock and Chambolle [82]. In particular, we apply 10 iterations of Ruiz scaling and then apply the Pock-Chambolle scaling. Appendix C.5 of Applegate et al. [9] shows that PDLP’s diagonal preconditioning outperforms either Pock-Chambolle or Ruiz individually.

Infeasibility detection. Inspired by [11], PDLP periodically checks whether any of three sequences—the difference of consecutive iterates, the normalized iterates, or the normalized running average—can serve as approximate certificates of infeasibility. We evaluate running averages starting from the most recent restart. This allows PDLP to detect infeasibility with little additional overhead. Notably, verifying all three sequences is often faster than relying on just one, since their convergence speed varies with the problem’s geometry [11].

Theoretical guarantees for the above enhancements. While PDLP’s enhancements are motivated by theory, some of them may not preserve theoretical guarantees: we do not have a proof of convergence for the adaptive step size rule or primal weight update scheme. However, Applegate et al. [10] showed in a simpler setting with fixed primal weight and step size that adaptive restarts preserve convergence guarantees. Similarly, the theoretical guarantees for infeasibility detection [11] do not account for restarts.

4 Quickly finding approximately optimal but feasible solutions

In integer programming, approximately optimal but feasible solutions are typically acceptable. For example, branch-and-bound methods are commonly stopped as soon as the optimality gap falls below a predefined threshold (e.g., 1%) to avoid excessive computation time while still delivering high-quality solutions [76]. This pragmatic termination criterion motivates a new heuristic we develop, called feasibility polishing, that enables first-order methods to similarly find solutions with extremely small feasibility violations and moderate (e.g., 1%) duality gaps.

There are two key insights behind our heuristic:

1. **Restarted PDHG tends to converge faster for feasibility problems than for optimality problems.** By a feasibility problem, we mean a problem with no objective, i.e.,

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimize}} && 0 \\ & \text{subject to:} && \ell_c \leq Ax \leq u_c \\ & && \ell_v \leq x \leq u_v. \end{aligned} \tag{6}$$

Empirically, these problems can be orders of magnitude faster for PDHG to solve compared to the original linear program (with nonzero objective).

2. **Given a starting solution, restarted PDHG converges to a *nearby* optimal solution.** PDHG has the property that the distance (in terms of the PDHG norm) from the iterates to any optimal

⁴Diagonal preconditioning is equivalent to changing to a weighted ℓ_2 norm in the proximal step of PDHG (weight defined by D_2 and D_1 for the primal and dual respectively). Pock and Chambolle use this weighted norm perspective.

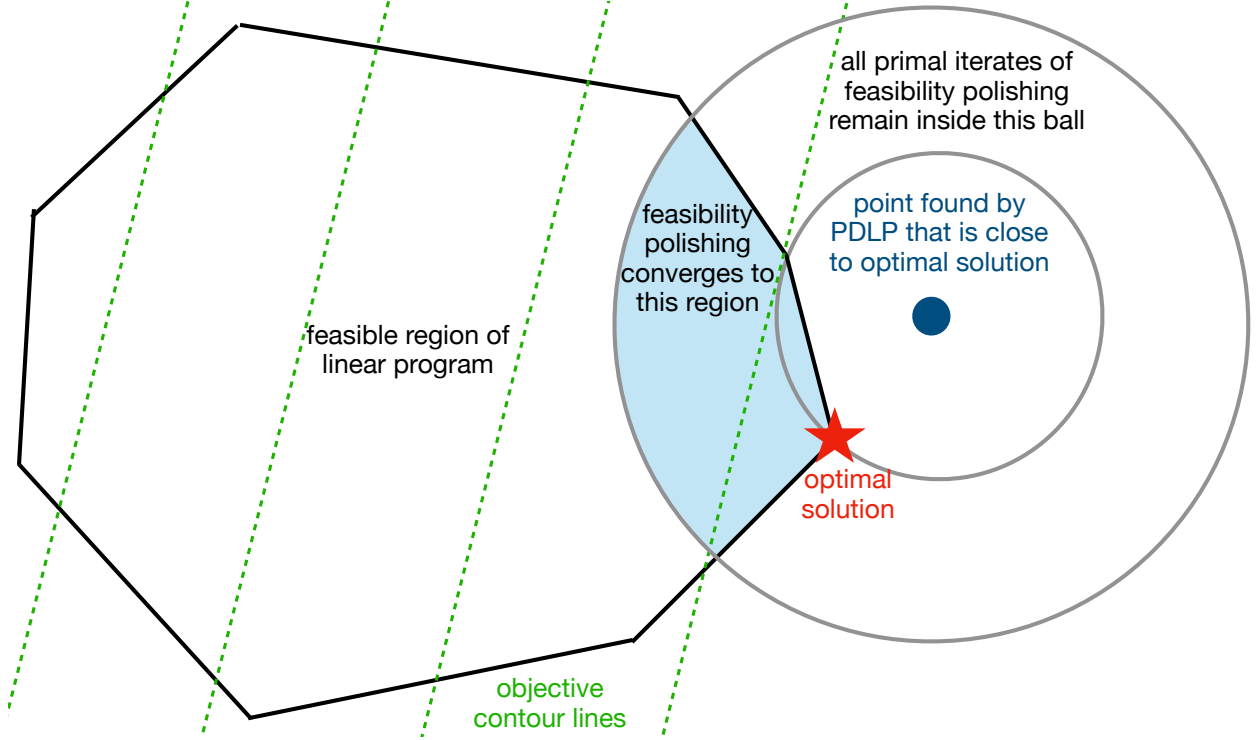


Figure 1: Visualization of feasibility polishing.

solution is nonincreasing [53]. The same property also holds for restarted PDHG (e.g., see Proposition 9 of Applegate et al. [10]). Consequently, one can show, if we warm-start restarted PDHG on (6) from a primal solution obtained by applying PDLF to the original problem (and set the starting dual iterate to zero, which is trivially dual feasible for (6)), then restarted PDHG will find a nearby primal feasible solution.

Thus, if we start from a near-optimal solution (i.e., from PDLF) as a warm-start for restarted PDHG on the primal feasibility problem (6), then the result will be a primal solution obtaining a similar objective value which satisfies tight feasibility tolerances. This idea is visualized in Figure 1.

We can repeat this heuristic argument on the dual feasibility problem

$$\begin{aligned}
 & \underset{y \in \mathbb{R}^m, r \in \mathbb{R}^n}{\text{maximize}} && 0 \\
 & \text{subject to:} && c - A^\top y = r \\
 & && y \in \mathcal{Y} \\
 & && r \in \mathcal{R}.
 \end{aligned} \tag{7}$$

to obtain a primal and dual solution that are almost exactly feasible with a small, but relatively larger, optimality gap.

Appendix C gives a theoretical basis for these two ideas. In particular, we show that PDHG with a fixed-frequency restart scheme and fixed step size can achieve an $O(\beta/H(A) \log(1/\epsilon))$ iteration bound on LP instances, where β is the largest singular value of the constraint matrix A and $H(A)$ is its Hoffman constant [55]. In other words, the new theory shows a runtime bound that is constant and independent of the right-hand side or objective vector. In contrast, [10] shows an $O(\beta/H(K) \log(1/\epsilon))$ iteration bound for restarted PDHG on general linear programs, where K is a matrix containing the A matrix, objective vector, and right-hand side. Moreover, some dependence on the right-hand side and/or objective vector is fundamental to restarted PDHG: indeed, there are examples where $H(A) = 1$ but restarted PDHG is slow to converge (see [54, Table 1]). We also show that our polishing algorithm will converge to a nearby feasible solution.

We note that similar arguments could be applied to other first-order methods, for example, replacing PDHG with AGD and minimizing $\|Ax - b\|^2$ s.t. $x \geq 0$ or using an adaptive restart scheme (following the techniques of [10]). As our main focus is computational, for simplicity we analyze the setting of PDHG with fixed frequency restarts.

These two insights are employed to design Algorithm 4. In particular, Algorithm 4 strives for a solution that satisfies extremely tight feasibility tolerances (e.g., 10^{-8} relative tolerances) but with a moderately large user-defined tolerance $\epsilon_{\text{rel-gap}} > 0$ on the relative duality gap, i.e.,

$$\text{RELATIVE-GAP} := \frac{|c^\top x + p(-y; \ell_c, u_c) + p(-r; \ell_v, u_v)|}{|c^\top x| + |p(-y; \ell_c, u_c) + p(-r; \ell_v, u_v)|} \leq \epsilon_{\text{rel-gap}}, \quad (8)$$

e.g., with $\epsilon_{\text{rel-gap}} = 10^{-2}$. While this is a somewhat nonstandard relative gap definition (i.e., more typical would be a max over the two terms in the denominator), we use this choice for legacy reasons.

Algorithm 4: Feasibility polishing algorithm.

1. Run PDLP on the LP instance. After completing k iterations, for $k = 100, 200, 400, 800, \dots$, if the relative duality gap is less than $\epsilon_{\text{rel-gap}}$, pause “normal” PDLP iterations. Let (x^k, y^k) be the average iterate (since the last restart) after that k -th iteration.
2. Run PDLP on the primal feasibility problem, i.e., Equation (6), starting from $(x^k, 0)$ with the initial primal weight and step size equal to the current values from Step 1. Run for $k/8$ iterations or until the primal constraint violation is less than the termination tolerance. In the former case, return to Step 1, continuing “normal” PDLP iterations. Otherwise, let $\tilde{x}$ be the output primal solution.
3. Run PDLP on the dual feasibility problem, i.e., Equation (7), starting from $(0, y^k)$ with the initial primal weight and step size equal to the current values from Step 1. Run for $k/8$ iterations or until the dual constraint violation is less than the termination tolerance. Let $\tilde{y}$ be the output dual solution.
4. Check if $(\tilde{x}, \tilde{y})$ meets the termination criteria; if not, return to Step 1, resuming “normal” PDLP iterations.

Note that operations that are triggered periodically based on iteration counts (such as termination checks, adaptive restarts — see the paragraph “Periodic checks of restarts and termination criteria” in Section 3) take place in Steps 1, 2, and 3 using the iteration count associated with the current Step.

5 Implementation

The PDLP implementation we refer to in this work is a C++ implementation that is distributed as part of Google’s open-source OR-Tools library. This implementation of PDLP was first released with OR-Tools version 9.3 in March 2022. In this section, we briefly discuss the motivations and notable differences between this code and the Julia-based prototype used in the conference version of this work.

The primary motivation for a C++ implementation was the desire to integrate PDLP into production applications at Google, where concerns about technical uniformity make C++ more practical to deploy than Julia. Secondly, the new implementation provided an opportunity to improve efficiency and generality, given that the Julia implementation was intended as a research prototype.

In addition to its improved accessibility thanks to its integration within both the OR-Tools framework and external modeling systems such as CVXPY [35], the C++ version has the following notable differences over the Julia code:

Feasibility polishing. The Julia code did not have a feasibility polishing routine.

Multithreading. The implementation parallelizes the computation of projections, norms, distances, and sparse matrix-dense vector multiplications by the constraint matrix $x \mapsto Ax$ and its transpose $y \mapsto A^\top y$. Any fixed number of threads is supported. To parallelize dense vector and vector-vector operations, it divides the vector into (approximately) equal sized shards, and performs each shard’s operations independently in

parallel (using Eigen for the per-shard operation). To parallelize sparse matrix-dense vector multiplications, it divides the sparse matrix by rows into shards with (approximately) the same number of nonzeros, and performs each shard’s product independently in parallel (using Eigen’s sparse matrix-dense vector product for the per-shard operation). Each shard is then handled independently and then combined⁵. By default, the number of shards is taken to be four times the number of threads.

More general LP form. As mentioned in Section 2, we consider a more general LP format, commonly adopted by LP solvers, where inequalities are permitted to have both lower and upper bounds. Equalities are specified by setting the lower bounds equal to the upper bounds. The C++ code supports this formulation, in contrast to the Julia code that supports only single-sided inequalities and equalities.

Presolve. The C++ implementation offers integrated presolving, in contrast with the conference version [9] where, for research purposes, we manually applied an external presolver. We did not develop new presolving routines; rather, we reused them from GLOP, the simplex-based LP solver included in OR-Tools. As a result of reusing GLOP’s presolving, the presolving code is single-threaded and not necessarily suitable for the largest scale of instances that PDLP is otherwise capable of handling. Thus, due to the size of the instances in the numerical experiments, we do not enable presolve in this work.

6 A New Collection of Large-Scale Instances

As part of this work, we developed and released a new test suite of large-scale LP instances. The set of problems is designed to be challenging for, if not beyond the capabilities of, today’s state-of-the-art solvers, yet small enough to fit in memory of top-end machines. The smallest problem in our new test suite has more than 125 million nonzeros, while the largest has over 6.3 billion. The collection is intended to be both realistic and diverse, but we also acknowledge that this collection is not comprehensive. The eleven LP instances are detailed in Table 1 and Table 2. Many of these problem instances were generated by the authors for the purposes of this project. There are few publicly available test instances of such a large scale, and it is our hope that these instances will be useful for other researchers.

All these instances were rescaled such that

- (i) The objective vector has values -1 , 0 , or 1 .
- (ii) The right-hand side has values $-\infty$, -1 , 0 , 1 , or ∞ , except in the case that a constraint has two nonzero, finite, nonmatching lower and upper bounds. In this case, we rescale such that the maximum absolute value of the right hand side is one.

This rescaling is implemented in the *rescale_instance* function in *utils.jl*.

These instances (after scaling) are available for download at

<https://www.oliverhinder.com/large-scale-lp-problems>

and some have generators available along with detailed descriptions of the problems in the repository

<https://github.com/ohinder/large-scale-LP-test-problems>

as indicated in the right column of Table 1. Although, all the instances in this *repository* are randomly generated (i.e., those marked with full details available), great effort was made by the authors to make this generation as realistic as possible, coupled with clearly documented reasoning.

⁵The algorithm is deterministic given a fixed number of shards. In each threaded operation, the per-shard operations are fully independent, so execution order doesn’t matter, and the per-shard results are processed sequentially (and deterministically). Changing the number of shards changes the order of the floating-point operations, and can result in different round-off errors.

Instance name	Brief description	Full details available?
design-match	Randomly generated statistical matching problem with covariate balancing constraints for making causal inference on observational data. Loosely inspired by Zubizarreta [100].	Yes
tsp-gaia-10m	An LP lower bound for a travelling salesman problem. The underlying TSP instance [32] is the 10 million stars nearest to Earth in the Gaia Data Release 2 star catalog [23]. The variables correspond to 60,601,996 edges heuristically chosen to minimize the reduced-cost penalty when pricing the edges from the complete graph. The constraints are the 10,000,000 degree constraints and 7,016,681 cutting planes collected from extensive runs of the Concorde TSP solver [8] on sub-instances.	No
tsp-gaia-100m	Same as tsp-gaia-10m, except using the 100 million stars nearest to Earth [31], with 1,184,557,727 edges, 100,000,000 degree constraints, and 62,934,799 cutting planes.	No
heat-source-easy	This problem considers a material with several randomly located heat sources. Given temperature measurements at various parts of the material and a list of possible locations of the heat sources, the aim is to recover the temperature distribution of the material. This is a PDE constrained optimization problem with linear partial differential equation constraints.	Yes
heat-source-hard	Same as heat-source-easy but significantly fewer measurement locations, more heat sources and more possible heat source locations. This makes the problem harder from a recovery standpoint. Moreover, we find this problem is much harder for first-order methods than heat-source-easy.	Yes
production-inventory	A robust production-inventory problem [16]. Data is randomly generated.	Yes
qap-tho-150	An LP relaxation [4] of the tho150 quadratic assignment problem from QAPLIB [3]. Data is randomly generated.	Yes
qap-wil-100	An LP relaxation [4] of the wil100 quadratic assignment problem from QAPLIB [3]. Data is randomly generated.	Yes
world-shipping	An LP relaxation of a MIP formulation of the LINERLIB [22] World instance.	No
mediterranean-shipping	An LP relaxation of a MIP formulation of the LINERLIB [22] Mediterranean instance.	No
supply-chain	A multicommodity flow problem motivated by optimizing the supply chain of a large retailer. Data is randomly generated.	Yes

Table 1: Descriptions of the LP problem instances.

7 Numerical experiments

In this section we present numerical results on a new test suite of large-scale LP instances that we have developed and released as part of this work. Our goal is to show that first-order solvers, in particular PDLF with feasibility polishing, can provide feasible solutions to tight tolerances that are close-to-optimal, in reasonable time and without requiring enormous amounts of memory. We will compare PDLF with and without feasibility polishing to three commercial state-of-the-art Gurobi solvers: a barrier (i.e., interior point) method, and primal and dual simplex methods.

We also provide some supplementary experiments in Section F on smaller test problems. These experiments (i) demonstrate the value of feasibility polishing on the Mittelmann benchmark and report the performance of PDLF in detecting infeasibility on a standard collection of primal infeasible problems. Not surprisingly we find that Gurobi is much faster than PDLF on most of these relatively small instances.

problem name	variables	constraints	nonzeros
design-match	40,000,000	22,000,135	2,760,000,000
tsp-gaia-10m	60,601,996	17,016,681	475,701,996
tsp-gaia-100m	1,184,557,727	162,934,799	6,337,834,450
heat-source-easy	31,628,008	15,625,000	125,000,000
heat-source-hard	31,628,008	15,625,000	125,000,000
production-inventory	18,270,600	4,650,850	500,049,700
qap-tho-150	249,783,750	6,705,300	1,005,795,000
qap-wil-100	49,015,000	1,980,200	198,020,000
world-shipping	228,867,510	15,304,282	688,658,522
mediterranean-shipping	208,479,461	7,490,593	628,927,462
supply-chain	201,000,100	2,210,100	403,000,100

Table 2: Problem instance sizes.

Detailed CSVs of the results and log files for all experiments can also be found at:

<https://www.oliverhinder.com/large-scale-lp-problems>.

We use the PDLP implementation in OR-Tools version 9.10. Instructions on how to run PDLP to reproduce our results can be found at

<https://github.com/ohinder/large-scale-LP-test-problems/blob/main/SOLVER-INSTRUCTIONS.md>.

Note: we do not compare against other first-order methods or GPU implementations. We do not compare against first-order methods, because in the conference version of this paper [9] we already tested against SCS, a state-of-the-art first-order method. In these tests, PDLP was $19\times$ faster on NETLIB [9, Table 24] and $6.3\times$ faster on MIPLIB 2017 relaxations, and thus we do not anticipate it being competitive in our tests. We do not compare against GPU implementations because the release of the PDLP C++ code in 2022 pre-dated the GPU implementation of PDLP [66]. Additionally, some of the instances that we test exceed the RAM limit of a single GPU. We expect that a GPU implementation would outperform the OR-Tools code in wall clock time for instances that fit in the GPU’s memory.

Machines used We used the Pitt computing cluster [2] to perform our experiments. There were two types of machines that we used: (i) an AMD EPYC 7302 (Rome) processor with 16 cores (32 threads) and 256 GB of memory and (ii) an Intel Xeon Platinum 8352Y (Ice Lake) processor with 32 cores (64 threads) and 1 TB of memory. The high memory machine used two and a half times more computing credits per hour so to limit its use we first ran a solver on an instance on the machine with 256 GB of memory and then only switched to the 1 TB machine if insufficient memory was available. We estimate that our experiments would have cost around \$20,000 to run on Google cloud (excluding the cost of building and rescaling the instances). We ran PDLP with 16 threads on the 256 GB machine and 32 threads on the 1 TB machine. For Gurobi, we left the thread parameter at its default value, which means that it will automatically use as many threads as the machine has up to the number of virtual processors available in the machine [1].

Multithreading performance of PDLP From Figure 2 we can see that PDLP enjoys significant speed-up from more threads on large instances. With 32 threads PDLP has an almost $8\times$ speed-up on the design-match instance, one of the largest in the set. This speed-up comes from parallelizing the sparse matrix-vector operation and vector-vector operations. For smaller instances, the overall speed-up can be much less, in fact, more threads can even be counterproductive [66, Table 8]. We have identified two areas to improve our multithreading performance: thread synchronization and inefficient use of memory bandwidth. Implementing ideas from the literature on multithreaded sparse matrix-vector (SpMV) operations (e.g., [24, 58]) would lead to further improvements.

Gurobi We use Gurobi version 11.0.2 with crossover disabled.

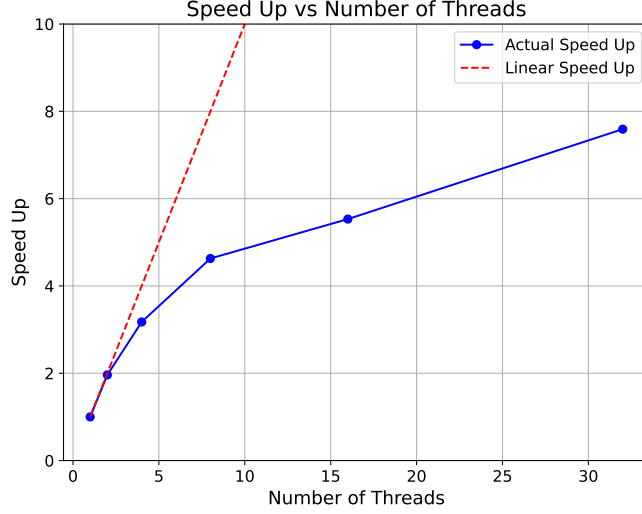


Figure 2: Speed up versus number of threads for PDLP on the design-match linear program on an AMD EPYC 7302 (Rome) processor with 16 cores (32 threads). We recorded the wall clock time to run 10,000 iterations with feasibility polishing disabled (1 measurement per number of threads).

Termination criteria To test feasibility polishing, our experiments do not use the default PDLP termination criteria. Instead, we use a tighter criterion for feasibility and a looser criterion for the optimality gap. In particular, in our experiments PDLP terminates when the maximum ℓ_∞ norms of the primal and dual residuals are less than 10^{-8} , i.e., $x \in \mathcal{X}$, $y \in \mathcal{Y}$, $r \in \mathcal{R}$,

$$\ell_c - 10^{-8} \leq Ax \leq u_c + 10^{-8}, \quad \|c - A^\top y - r\|_\infty \leq 10^{-8}, \quad (9)$$

and the relative duality gap is less than 1%, i.e.,

$$\frac{|c^\top x + p(-y; \ell_c, u_c) + p(-r; \ell_v, u_v)|}{|c^\top x| + |p(-y; \ell_c, u_c) + p(-r; \ell_v, u_v)|} \leq 10^{-2}.$$

Recall, however, that all linear programs have been scaled as per Section 6 which impacts the termination criteria. Specifically, Equation (9) *translated to the original linear programs* becomes equivalent to

$$\ell_c - 10^{-8}q \leq Ax \leq u_c + 10^{-8}q \quad \text{where} \quad q_i := \begin{cases} |(\ell_c)_i| & (\ell_c)_i \notin \{-\infty, 0\}, \quad (u_c)_i \in \{0, \infty\} \\ |(u_c)_i| & (\ell_c)_i \in \{-\infty, 0\}, \quad (u_c)_i \notin \{0, \infty\} \\ \max\{|(\ell_c)_i|, |(u_c)_i|\} & (\ell_c)_i \notin \{-\infty, 0\}, \quad (u_c)_i \notin \{0, \infty\} \\ 1 & (\ell_c)_i \in \{-\infty, 0\}, \quad (u_c)_i \in \{0, \infty\} \end{cases} \quad (10a)$$

and

$$|(c - A^\top y - r)_i| \leq 10^{-8} \times \begin{cases} |c_i| & c_i \neq 0 \\ 1 & \text{otherwise.} \end{cases} \quad (10b)$$

Finally, we include the infeasibility termination criteria in Section D.

We used the default Gurobi termination criteria, which after presolve and their own custom scaling, has an ℓ_∞ tolerance on the primal and dual residuals of 10^{-6} , and objective gap tolerance of 10^{-8} . Note that we do not believe that making Gurobi's termination criteria match ours would make a significant difference to the results. In particular, Gurobi's barrier implementation mostly experiences OOM errors, and so is unaffected by the termination criteria; and simplex methods only find a primal and dual feasible solution at the optimum.

Results Table 3 shows the time in **hours** taken per instance for each method to solve the problems and whether the method required a high memory machine or ran out of memory (shading the cell red). From this table one can see that Gurobi barrier only solves three instances because it hits memory limits in all but three instances, whereas PDLP never exceeds 1 TB of memory and only in one instance requires an excess of 256 GB. Gurobi’s primal and dual simplex is competitive with PDLP in terms of memory requirements but slow: combined primal and dual simplex only solves four instances within the time limit of 144 hours. Finally, we see that feasibility polishing is extremely effective: PDLP without polishing only solves two instances, PDLP with polishing solves eight of eleven instances. Table 4 shows that the relative duality gap achieved by PDLP with feasibility polishing is typically much better than the 1% threshold that we targeted. In particular, in all cases it obtains a duality gap of less than 0.4% and other than the two shipping instances the duality gap is always less than 0.1%. Of course, these duality gaps are much worse than what Gurobi would achieve if it solved the problem (recall 10^{-8} is the default absolute optimality gap for which Gurobi barrier terminates) but for many applications this may be satisfactory.

Table 3 shows that both PDLP and Gurobi had errors. Both of Gurobi’s errors gave the error code 10025. We believe this is a bug in Gurobi as the Gurobi error codes [49] state that the ‘user has called a query routine on a model with more than 2 billion non-zero entries’ even though we did not call a query routine. We have made Gurobi aware of this issue. Both of PDLP’s errors were numerical errors that occurred within a few hundred iterations.

Instance name	PDLP		Gurobi		
	Without polishing	With polishing	Barrier	Dual	Primal
design-match	68.0	9.3	OOM	75.3	32.4
tsp-gaia-10m	TIME	3.0	28.1	90.1	40.2
tsp-gaia-100m	TIME	21.1	OOM	OOM	OOM
heat-source-easy	TIME	60.0	OOM	ERR	TIME
heat-source-hard	TIME	TIME	OOM	ERR	TIME
production-inventory	TIME	TIME	5.8	TIME	43.7
qap-tho-150	ERR	ERR	OOM	TIME	TIME
qap-wil-100	44.4	0.3	OOM	TIME	TIME
world-shipping	TIME	53.8	OOM	TIME	TIME
mediterranean-shipping	TIME	32.6	OOM	TIME	TIME
supply-chain	TIME	18.9	2.5	4.2	TIME

Table 3: Comparison of solvers in terms of the number of **hours** to solve instances (excluding time to read the instance). Termination criteria for Gurobi was set as default; termination criteria for PDLP was 10^{-8} maximum violation on primal and dual feasibility but 10^{-2} on the relative duality gap. Cells shaded red indicate runs that required the 1 TB RAM machine. TIME indicates that the solver exceeded 144 hours. OOM stands for ‘out of memory’, meaning the solver exceeded the 1 TB RAM limit. ERR denotes that there was an error during the solve.

8 Conclusion

In this work, we demonstrated that feasibility polishing significantly improves the ability of PDLP to find solutions with tight feasibility tolerances (both primal and dual) but more relaxed optimality tolerances. Practitioners would obviously prefer solutions that satisfy tight feasibility and optimality tolerances—solutions which are reliably provided by interior point methods and the simplex method on moderately large instances. However, the standard strategy of relaxing both feasibility and optimality tolerances to allow first-order methods to produce solutions within a reasonable timeframe [75] is undesirable: moderate feasibility violations can give unexpectedly catastrophic solutions to the true underlying problem. This issue is not merely hypothetical; it occurs in the ‘irish-electricity’ instance as discussed in the introduction. Moreover, ruling out this issue requires a solution with tight feasibility tolerances, which defeats the purpose of using a first-order method for a fast, inaccurate solve.

Instance name	Primal objective	Absolute gap	Relative gap (%)
design-match	2.87×10^6	5.53×10^{-2}	0.0000001
tsp-gaia-10m	1.3×10^8	3.01×10^3	0.0012
tsp-gaia-100m	1.94×10^9	5.82×10^4	0.0015
heat-source-easy	5.16×10^7	9.06×10^{-4}	0.00000000088
qap-wil-100	2.21×10^5	1.14×10^2	0.026
world-shipping	-1.18×10^5	3.7×10^2	0.16
mediterranean-shipping	-1.07×10^3	6.95×10^0	0.33
supply-chain	1.08×10^8	6.67×10^3	0.0031

Table 4: Objectives and duality gaps produced from PDLP with feasibility polishing for instances that successfully terminated. Note the relative gap is computed with the somewhat non-standard formula given in Equation (8).

We also showed, on a new collection of very large linear programs and a machine with 1TB of RAM, that PDLP with feasibility polishing can find solutions with extremely small feasibility violations and moderate duality gaps for several problems that Gurobi is unable to solve within six days. While we expect our work to expand the number of large-scale linear programs that can be solved, it is important to emphasize the inherent limitations of first-order methods: they are unlikely to ever completely replace second-order methods. In particular, one can construct linear systems with binary entries and condition number exponential in the dimension [6]. Such linear systems are unlikely to be solved efficiently by first-order methods, which in the worst case require a number of iterations polynomial in the condition number to obtain high-accuracy solutions.

Nonetheless, our numerical results illustrate that large sets of practical linear programs can be effectively tackled with first-order methods. Indeed, when constraint matrices are totally unimodular [54] or have small circuit imbalance [29], first-order methods provably require only a polynomial (in the problem dimensions) number of iterations to find solutions satisfying the optimality conditions with tight tolerances. Relaxing the termination criteria to require only tight feasibility tolerances with moderate duality gap tolerances will likely expand this set even further.

Acknowledgements

We thank Ed Klotz for his helpful discussion regarding the Irish Electricity instance and his help clarifying the termination criteria of Gurobi. We also thank Joey Huchette for his feedback on an early draft of the paper.

This research was supported in part by the University of Pittsburgh Center for Research Computing, RRID:SCR_022735, through the resources provided. Specifically, this work used the SMP cluster, which is supported by NSF award number OAC-2117681.

Author contributions

Authors are ordered alphabetically. All authors contributed to writing, and regular discussion in meetings. The listed contributions span both the conference and journal papers.

DA and WS jointly designed the feasibility polishing implementation, and implemented most of the C++ code (except for the parts specifically mentioned elsewhere).

DA led the project after ML left Google, and generated the large-scale TSP instances which were our first large-scale test problems.

MD analyzed and implemented infeasibility detection features in both Julia and C++. He implemented and evaluated our step size choice against Malitsky and Pock's. He also proofread and evaluated the large instances designed by OH.

OH devised the restart and primal weight scheme, implementing them in FirstOrderLp.jl; he proposed, and demonstrated the feasibility polishing concept. He also created the design-match, supply-chain and heat-source test problems. DA and WS ran preliminary experiments for this paper and OH ran the final experiments.

HL's internship provided the impetus and initial code to pursue the idea of using first-order methods for LP. After his internship, HL, jointly with BOD, proposed the use of PDHG as the base algorithm. He designed and implemented the preconditioning scheme in both the FirstOrderLp.jl and C++ code. He also contributed to the development of the theory for restart strategies and infeasibility detection which influenced the implementation, and he implemented the production-inventory problem.

ML led the project while he was at Google, recruited and supervised HL, OH, and MD for their work on the project that was conducted at Google, contributed significantly to the FirstOrderLp.jl and OR-Tools implementations, created a framework for running large-scale experiments, suggested many aspects of the design of the numerical experiments, pushed forward the presolve enhancement, and led the open sourcing effort.

BOD, jointly with HL, proposed the usage of PDHG as the core algorithm to apply to LP. He also provided advice and feedback on the restarting techniques and the usage of SCS, and helped with data analysis and figure generation for the NeurIPS paper.

WS performed the initial experiments demonstrating the effectiveness of mirror-prox and PDHG for LP including some initial heuristics that inspired the primal weight scheme and restarts, and designed and implemented the sharded parallelism mechanism and the adaptive step size scheme.

References

- [1] Gurobi optimizer reference manual - threads parameter. <https://www.gurobi.com/documentation/current/refman/threads.html#parameter:Threads>, 2024. Accessed: 2024-09-14.
- [2] Center for research computing. <https://crc.pitt.edu/>, 2024. Accessed: 2024-09-14.
- [3] QAPLIB problem instances and solutions. <https://coral.ise.lehigh.edu/data-sets/qaplib/qaplib-problem-instances-and-solutions/>, 2024. Accessed: 2024-09-14.

- [4] Warren P. Adams and Terri A. Johnson. Improved linear programming-based lower bounds for the quadratic assignment problem. In Panos Pardalos and Henry Wolkowicz, editors, *Quadratic Assignment and Related Problems*, volume 16 of *DIMACS Series on Discrete Mathematics and Theoretical Computer Science*, pages 43–75. AMS, Providence, RI, 1994.
- [5] Ahmet Alacaoglu, Olivier Fercoq, and Volkan Cevher. On the convergence of stochastic primal-dual hybrid gradient. *SIAM Journal on Optimization*, 2022.
- [6] Noga Alon and Văn H Vū. Anti-Hadamard matrices, coin weighing, threshold gates, and indecomposable hypergraphs. *Journal of Combinatorial Theory, Series A*, 79(1):133–160, 1997.
- [7] Roberto Andreani, Ernesto G Birgin, José Mario Martínez, and María Laura Schuverdt. On augmented Lagrangian methods with general lower-level constraints. *SIAM Journal on Optimization*, 18(4):1286–1309, 2008.
- [8] David Applegate, Robert Bixby, Vašek Chvátal, and William Cook. Concorde TSP solver. <https://www.math.uwaterloo.ca/tsp/concorde.html>, 2020.
- [9] David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O’Donoghue, and Warren Schudy. Practical large-scale linear programming using primal-dual hybrid gradient. *Advances in Neural Information Processing Systems*, 34:20243–20257, 2021.
- [10] David Applegate, Oliver Hinder, Haihao Lu, and Miles Lubin. Faster first-order primal-dual methods for linear programming using restarts and sharpness. *Mathematical Programming*, 201(1):133–184, 2023.
- [11] David Applegate, Mateo Díaz, Haihao Lu, and Miles Lubin. Infeasibility detection with primal-dual hybrid gradient for large-scale linear programming. *SIAM Journal on Optimization*, 34(1):459–484, 2024.
- [12] Kinjal Basu, Amol Ghoting, Rahul Mazumder, and Yao Pan. ECLIPSE: An extreme-scale linear program solver for web-applications. In *International Conference on Machine Learning*, pages 704–714. PMLR, 2020.
- [13] Heinz H Bauschke and Patrick L Combettes. *Convex analysis and monotone operator theory in Hilbert spaces*, volume 408. Springer, 2nd edition, 2017.
- [14] Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- [15] Stephen R Becker, Emmanuel J Candès, and Michael C Grant. Templates for convex cone problems with applications to sparse signal recovery. *Mathematical Programming Computation*, 3(3):165–218, 2011.
- [16] Aharon Ben-Tal, Alexander Goryashko, Elana Guslitzer, and Arkadi Nemirovski. Adjustable robust solutions of uncertain linear programs. *Mathematical Programming*, 99(2):351–376, 2004.
- [17] Ernesto G Birgin and José Mario Martínez. Large-scale active-set box-constrained optimization method with spectral projected gradients. *Computational Optimization and Applications*, 23(1):101–125, 2002.
- [18] Nicolas Blin. Accelerate large linear programming problems with NVIDIA cuOpt. <https://developer.nvidia.com/blog/accelerate-large-linear-programming-problems-with-nvidia-cuopt/>, 2024.
- [19] Stephen Boyd and Lieven Vandenbergh. *Convex optimization*. Cambridge university press, 2004.
- [20] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends® in Machine learning*, 3(1):1–122, 2011.

- [21] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Computer networks and ISDN systems*, 30(1-7):107–117, 1998.
- [22] Berit Brouer. LINERLIB. <https://github.com/blouf/LINERLIB>, 2013.
- [23] A. G. A. Brown, A. Vallenari, T. Prusti, et al. Gaia data release 2: Summary of the contents and survey properties. *Astronomy & Astrophysics*, 616:A1, August 2018. doi: 10.1051/0004-6361/201833051.
- [24] Aydin Buluç, Samuel Williams, Leonid Oliker, and James Demmel. Reduced-bandwidth multithreaded algorithms for sparse matrix-vector multiplication. In *2011 IEEE International Parallel & Distributed Processing Symposium*, pages 721–733. IEEE, 2011.
- [25] Paula Carroll, Damian Flynn, Bernard Fortz, and Alex Melhorn. Sub-hour unit commitment MILP model with benchmark problem instances. In *Computational Science and Its Applications–ICCSA 2017: 17th International Conference, Trieste, Italy, July 3-6, 2017, Proceedings, Part II 17*, pages 635–651. Springer, 2017.
- [26] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011.
- [27] Antonin Chambolle and Thomas Pock. On the ergodic convergence rates of a first-order primal–dual algorithm. *Mathematical Programming*, 159(1):253–287, 2016.
- [28] Antonin Chambolle, Matthias J Ehrhardt, Peter Richtárik, and Carola-Bibiane Schonlieb. Stochastic primal-dual hybrid gradient algorithm with arbitrary sampling and imaging applications. *SIAM Journal on Optimization*, 28(4):2783–2808, 2018.
- [29] Richard Cole, Christoph Hertrich, Yixin Tao, and László A Végh. A first order method for linear programming parameterized by circuit imbalance. *Mathematical Programming*, pages 1–39, 2025.
- [30] Laurent Condat. A primal–dual splitting method for convex optimization involving Lipschitzian, proximable and linear composite terms. *Journal of Optimization Theory and Applications*, 158(2):460–479, 2013.
- [31] William Cook. 100,000,000 stars. <https://www.math.uwaterloo.ca/tsp/star/star100m.html>, 2024.
- [32] William Cook. 10,000,000 stars. <https://www.math.uwaterloo.ca/tsp/star/star10m.html>, 2024.
- [33] Yiran Cui, Keiichi Morikuni, Takashi Tsuchiya, and Ken Hayami. Implementation of interior-point methods for LP based on Krylov subspace iterative solvers with inner-iteration preconditioning. *Computational Optimization and Applications*, 74(1):143–176, 2019.
- [34] Qi Deng, Qing Feng, Wenzhi Gao, Dongdong Ge, Bo Jiang, Yuntian Jiang, Jingsong Liu, Tianhao Liu, Chenyu Xue, Yinyu Ye, and Chuwen Zhang. An enhanced alternating direction method of multipliers-based interior point method for linear and conic optimization. *INFORMS Journal on Computing*, 37(2):338–359, 2025.
- [35] Steven Diamond and Stephen P. Boyd. CVXPY: A python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17, 2016.
- [36] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [37] Jonathan Eckstein and Dimitri P Bertsekas. An alternating direction method for linear programming. Technical Report LIDS-P-1967, Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, 1990.
- [38] Jonathan Eckstein and Dimitri P Bertsekas. On the Douglas–Rachford splitting method and the proximal point algorithm for maximal monotone operators. *Mathematical Programming*, 55(1-3):293–318, 1992.

- [39] Ernie Esser, Xiaoqun Zhang, and Tony F Chan. A general framework for a class of first order primal-dual algorithms for convex optimization in imaging science. *SIAM Journal on Imaging Sciences*, 3(4):1015–1046, 2010.
- [40] Kimon Fountoulakis, Jacek Gondzio, and Pavel Zhlobich. Matrix-free interior point method for compressed sensing problems. *Mathematical Programming Computation*, 6(1):1–31, 2014.
- [41] Michael Garstka, Mark Cannon, and Paul Goulart. COSMO: A conic operator splitting method for convex conic problems. *Journal of Optimization Theory and Applications*, 190(3):779–810, 2021.
- [42] David M Gay. Electronic mail distribution of linear programming test problems. *Mathematical Programming Society COAL Newsletter*, 13:10–12, 1985.
- [43] Dongdong Ge, Qi Huangfu, Zizhuo Wang, Jian Wu, and Yinyu Ye. Cardinal Optimizer (COPT) user guide. <https://guide.coap.online/copt/en-doc>, 2024.
- [44] Andrew Gilpin, Javier Peña, and Tuomas Sandholm. First-order algorithm with $\mathcal{O}(\ln(1/\epsilon))$ convergence for ϵ -equilibrium in two-person zero-sum games. *Mathematical Programming*, 133(1):279–298, 2012.
- [45] Tom Goldstein, Min Li, Xiaoming Yuan, Ernie Esser, and Richard Baraniuk. Adaptive primal-dual hybrid gradient methods for saddle-point problems. *arXiv:1305.0546*, 2013.
- [46] Tom Goldstein, Min Li, and Xiaoming Yuan. Adaptive primal-dual splitting methods for statistical learning and image processing. In *Advances in Neural Information Processing Systems*, pages 2089–2097, 2015.
- [47] Jacek Gondzio. Matrix-free interior point method. *Computational Optimization and Applications*, 51(2):457–480, 2012.
- [48] Jacek Gondzio, Spyridon Pougkakiotis, and John W Pearson. General-purpose preconditioning for regularized interior point methods. *Computational Optimization and Applications*, 83(3):727–757, 2022.
- [49] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual: Error Codes*, 2024. URL https://www.gurobi.com/documentation/current/refman/error_codes.html. Accessed: 2024-09-13.
- [50] Benjamin Halpern. Fixed points of nonexpanding maps. *Bulletin of the American Mathematical Society*, 1967.
- [51] Bingsheng He and Xiaoming Yuan. Convergence analysis of primal-dual algorithms for a saddle-point problem: from contraction perspective. *SIAM Journal on Imaging Sciences*, 5(1):119–149, 2012.
- [52] Bingsheng He and Xiaoming Yuan. On the $O(1/n)$ convergence rate of the Douglas–Rachford alternating direction method. *SIAM Journal on Numerical Analysis*, 50(2):700–709, 2012.
- [53] Bingsheng He, Yanfei You, and Xiaoming Yuan. On the convergence of primal-dual hybrid gradient algorithm. *SIAM Journal on Imaging Sciences*, 7(4):2526–2537, 2014.
- [54] Oliver Hinder. Worst-case analysis of restarted primal-dual hybrid gradient on totally unimodular linear programs. *Operations Research Letters*, 57:107199, 2024.
- [55] Alan J. Hoffman. On approximate solutions of systems of linear inequalities. *Journal of Research of the National Bureau of Standards*, 49:263–265, 1952.
- [56] Qi Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.
- [57] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

- [58] M Ozan Karsavuran, Kadir Akbudak, and Cevdet Aykanat. Locality-aware parallel sparse matrix-vector and matrix-transpose-vector multiplication on many-core processors. *IEEE Transactions on Parallel and Distributed Systems*, 27(6):1713–1726, 2015.
- [59] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [60] Guanghui Lan, Zhaosong Lu, and Renato D. C. Monteiro. Primal-dual first-order methods with $\mathcal{O}(1/\epsilon)$ iteration-complexity for cone programming. *Mathematical Programming*, 2011.
- [61] Tianyi Lin, Shiqian Ma, Yinyu Ye, and Shuzhong Zhang. An ADMM-based interior-point method for large-scale linear programming. *Optimization Methods and Software*, 36(2-3):389–424, 2021.
- [62] Haihao Lu and Jinwen Yang. On the infimal sub-differential size of primal-dual hybrid gradient method and beyond. *arXiv:2206.12061*, 2022.
- [63] Haihao Lu and Jinwen Yang. On a unified and simplified proof for the ergodic convergence rates of PPM, PDHG and ADMM. *arXiv:2305.02165*, 2023.
- [64] Haihao Lu and Jinwen Yang. Restarted Halpern PDHG for linear programming. *arXiv:2407.16144*, 2024.
- [65] Haihao Lu and Jinwen Yang. On the geometry and refined rate of primal–dual hybrid gradient for linear programming. *Mathematical Programming*, 212:349–387, 2025.
- [66] Haihao Lu and Jinwen Yang. cuPDLP.jl: A GPU implementation of restarted primal-dual hybrid gradient for linear programming in julia. *Operations Research*, 73(6):3440–3452, 2025.
- [67] Haihao Lu, Jinwen Yang, Haodong Hu, Qi Huangfu, Jinsong Liu, Tianhao Liu, Yinyu Ye, Chuwen Zhang, and Dongdong Ge. cuPDLP-C: A strengthened implementation of cuPDLP for linear programming by C language. *arXiv:2312.14832*, 2023.
- [68] István Maros. *Simplex-Based LP Solvers*. John Wiley & Sons, Ltd, 2011. ISBN 9780470400531.
- [69] H Brendan McMahan and Matthew Streeter. Adaptive bound optimization for online convex optimization. *arXiv:1002.4908*, 2010.
- [70] MIPLIB. Irish Electricity Instance - MIPLIB 2017, 2017. URL https://miplib.zib.de/instance_details_irish-electricity.html. Accessed: 2024-10-14.
- [71] Hans D Mittelmann. Decision tree for optimization software. <http://plato.asu.edu/guide.html>, 2021.
- [72] Ion Necoara, Yu Nesterov, and Francois Glineur. Linear convergence of first order methods for non-strongly convex optimization. *Mathematical Programming*, 2019.
- [73] Yurii Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27(2):372–376, 1983.
- [74] Brendan O’Donoghue. Operator splitting for a homogeneous embedding of the linear complementarity problem. *SIAM Journal on Optimization*, 31(3):1999–2023, 2021.
- [75] Brendan O’Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic optimization via operator splitting and homogeneous self-dual embedding. *Journal of Optimization Theory and Applications*, 169(3):1042–1068, 2016.
- [76] Gurobi Optimization. What is the MIPGap? <https://support.gurobi.com/hc/en-us/articles/8265539575953-What-is-the-MIPGap>. Accessed: 2024-10-07.
- [77] Daniel O’Connor and Lieven Vandenbergh. On the equivalence of the primal-dual hybrid gradient method and Douglas–Rachford splitting. *Mathematical Programming*, 179(1):85–108, 2020.

- [78] Brendan O’Donoghue and Emmanuel Candès. Adaptive restart for accelerated gradient schemes. *Foundations of computational mathematics*, 15(3):715–732, 2015.
- [79] Brendan O’Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. SCS: Splitting conic solver, version 3.27. <https://github.com/cvxgrp/scs>, November 2017.
- [80] Neal Parikh and Stephen Boyd. Proximal algorithms. *Foundations and Trends® in Optimization*, 1(3):127–239, 2014.
- [81] Laurent Perron and Vincent Furnon. OR-Tools, 2024. URL <https://developers.google.com/optimization/>.
- [82] Thomas Pock and Antonin Chambolle. Diagonal preconditioning for first order primal-dual algorithms in convex optimization. In *2011 International Conference on Computer Vision*, pages 1762–1769. IEEE, 2011.
- [83] Thomas Pock, Daniel Cremers, Horst Bischof, and Antonin Chambolle. An algorithm for minimizing the Mumford-Shah functional. In *2009 IEEE 12th International Conference on Computer Vision*, pages 1133–1140. IEEE, 2009.
- [84] James Renegar. A polynomial-time algorithm, based on Newton’s method, for linear programming. *Mathematical Programming*, 40(1):59–93, 1988.
- [85] James Renegar. Accelerated first-order methods for hyperbolic programming. *Mathematical Programming*, 173(1):1–35, 2019.
- [86] R Tyrrell Rockafellar. Monotone operators and the proximal point algorithm. *SIAM Journal on Control and Optimization*, 14(5):877–898, 1976.
- [87] Daniel Ruiz. A scaling algorithm to equilibrate both rows and columns norms in matrices. Technical report, CM-P00040415, 2001.
- [88] Ernest K Ryu and Stephen Boyd. Primer on monotone operator methods. *Appl. Comput. Math*, 15(1):3–43, 2016.
- [89] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.
- [90] Robert J. Vanderbei. *Problems in General Form*. Springer International Publishing, Cham, Switzerland, 2020.
- [91] Sinong Wang and Ness Shroff. A new alternating direction method for linear programming. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [92] Laurence A Wolsey. *Integer programming*. John Wiley & Sons, Hoboken, NJ, USA, 2021.
- [93] Margaret Wright. The interior-point revolution in optimization: history, recent developments, and lasting consequences. *Bulletin of the American Mathematical Society*, 42(1):39–56, 2005.
- [94] Zikai Xiong and Robert M Freund. The role of level-set geometry on the performance of PDHG for conic linear optimization. *arXiv:2406.01942*, 2024.
- [95] FICO Xpress. FICO Xpress Optimization Suite. 2014.
- [96] Tianbao Yang and Qihang Lin. RSG: Beating subgradient method without smoothness and strong convexity. *Journal of Machine Learning Research*, 19(1):236–268, 2018.
- [97] Yinyu Ye, Michael J Todd, and Shinji Mizuno. An $O(\sqrt{nL})$ -iteration homogeneous and self-dual linear programming algorithm. *Mathematics of Operations Research*, 19(1):53–67, 1994.

- [98] Filippo Zanetti and Jacek Gondzio. A new stopping criterion for Krylov solvers applied in interior point methods. *SIAM Journal on Scientific Computing*, 45(2):A703–A728, 2023.
- [99] Mingqiang Zhu and Tony Chan. An efficient primal-dual hybrid gradient algorithm for total variation image restoration. *UCLA CAM Report*, 34:8–34, 2008.
- [100] José R Zubizarreta. Using mixed integer programming for matching in an observational study of kidney failure after surgery. *Journal of the American Statistical Association*, 107(500):1360–1371, 2012.

A Reduced cost selection for PDLP

As discussed in Section 2, there are multiple choices for recovering the reduced costs when running PDLP. These choices impact convergence detection but not the sequence of iterates. A natural choice is to set

$$r \leftarrow \underset{\mathcal{R}}{\mathbf{proj}}(c - A^\top y);$$

however, an alternative that we found to be useful is to set

$$r_i \leftarrow \begin{cases} c_i - a_i^\top y & \text{if } c_i - a_i^\top y > 0 \text{ and } x_i - (\ell_v)_i \leq |x_i| \\ c_i - a_i^\top y & \text{if } c_i - a_i^\top y < 0 \text{ and } (u_v)_i - x_i \leq |x_i| \\ 0 & \text{otherwise} \end{cases}$$

where a_i is the i th column of A . This choice is more robust to artificially large bounds. For example, a relatively small residual reduced cost when a variable bound is extremely large, e.g., 10^{12} , could prevent convergence through its effect on the dual objective value. This more advanced choice is only used when we run PDLP *without* feasibility polishing.

B Derivation of Equation (3)

This section shows how Equation (3) follows from specializing Algorithm 1 of Chambolle and Pock [27] to our saddle-point formulation

$$\min_{x \in \mathcal{X}} \max_{y \in \mathcal{Y}} \mathcal{L}(x, y) = c^\top x - y^\top A x - p(y; -u_c, -\ell_c).$$

The standard saddle-point form of [27, Equation (2)] with $f(x) = 0$ and finite-dimensional spaces is

$$\min_{x \in \mathbb{R}^n} \max_{y \in \mathbb{R}^m} y^\top K x + g(x) - h^*(y),$$

where g and h are proper, lower semicontinuous, convex functions, and h^* is the convex conjugate of h . This saddle-point problem reduces to our formulation by setting

$$g(x) := c^\top x + I_{\mathcal{X}}(x), \quad h^*(y) := I_{\mathcal{Y}}(y) + p(y; -u_c, -\ell_c), \quad K := -A,$$

where for any set S and vector s we define the indicator function

$$I_S(s) := \begin{cases} \infty & \text{if } s \notin S, \\ 0 & \text{if } s \in S. \end{cases}$$

The PDHG algorithm, i.e., Algorithm 1 of Chambolle and Pock [27] with the Euclidean norm, takes the form

$$x^{k+1} = \underset{x}{\operatorname{argmin}} g(x) + (y^k)^\top K x + \frac{1}{2\tau} \|x - x^k\|_2^2, \tag{11a}$$

$$y^{k+1} = \underset{y}{\operatorname{argmin}} h^*(y) - y^\top K(2x^{k+1} - x^k) + \frac{1}{2\sigma} \|y - y^k\|_2^2, \tag{11b}$$

where $\tau, \sigma > 0$ satisfy $\tau\sigma\|K\|_2^2 < 1$.

Primal step. To see that Equation (11a) reduces to Equation (3a), observe that

$$\begin{aligned} x^{k+1} &= \underset{x}{\operatorname{argmin}} g(x) + (y^k)^\top K x + \frac{1}{2\tau} \|x - x^k\|_2^2 = \underset{x}{\operatorname{argmin}} c^\top x + I_{\mathcal{X}}(x) + (y^k)^\top (-A)x + \frac{1}{2\tau} \|x - x^k\|_2^2 \\ &= \underset{x \in \mathcal{X}}{\operatorname{argmin}} c^\top x - (y^k)^\top A x + \frac{1}{2\tau} \|x - x^k\|_2^2 = \underset{x \in \mathcal{X}}{\operatorname{argmin}} (c - A^\top y^k)^\top x + \frac{1}{2\tau} \|x - x^k\|_2^2 \\ &= \underset{\mathcal{X}}{\mathbf{proj}}(x^k - \tau(c - A^\top y^k)). \end{aligned}$$

Dual step. To see that Equation (11b) reduces to Equation (3b), observe that

$$\begin{aligned}
y^{k+1} &= \underset{y}{\operatorname{argmin}} h^*(y) - y^\top K(2x^{k+1} - x^k) + \frac{1}{2\sigma} \|y - y^k\|_2^2 \\
&= \underset{y}{\operatorname{argmin}} I_{\mathcal{Y}}(y) + p(y; -u_c, -\ell_c) - y^\top (-A)(2x^{k+1} - x^k) + \frac{1}{2\sigma} \|y - y^k\|_2^2 \\
&= \underset{y \in \mathcal{Y}}{\operatorname{argmin}} p(y; -u_c, -\ell_c) + y^\top A(2x^{k+1} - x^k) + \frac{1}{2\sigma} \|y - y^k\|_2^2 \\
&= \underset{y \in \mathcal{Y}}{\operatorname{argmin}} p(y; -u_c, -\ell_c) + \frac{1}{2\sigma} \|y - (y^k - \sigma A(2x^{k+1} - x^k))\|_2^2 \\
&= \mathbf{prox}_{\sigma p(\cdot; -u_c, -\ell_c)}(y^k - \sigma A(2x^{k+1} - x^k)). \tag{12}
\end{aligned}$$

The proximal operator of $\sigma p(\cdot; -u_c, -\ell_c)$ can be written as

$$\mathbf{prox}_{\sigma p(\cdot; -u_c, -\ell_c)}(v) = v - \sigma \mathbf{proj}_{[-u_c, -\ell_c]}(\sigma^{-1}v),$$

due to Moreau's decomposition, i.e., $v = \mathbf{prox}_f(v) + \mathbf{prox}_{f^*}(v)$ for $f(v) = \sigma p(v; -u_c, -\ell_c)$, and the fact that convex conjugate of f is $f^*(v) = I_{[-u_c, -\ell_c]}(\sigma^{-1}v)$. Substituting this expression into Equation (12) yields

$$\begin{aligned}
y^{k+1} &= y^k - \sigma A(2x^{k+1} - x^k) - \sigma \mathbf{proj}_{[-u_c, -\ell_c]}(\sigma^{-1}(y^k - \sigma A(2x^{k+1} - x^k))) \\
&= y^k - \sigma A(2x^{k+1} - x^k) - \sigma \mathbf{proj}_{[-u_c, -\ell_c]}(\sigma^{-1}y^k - A(2x^{k+1} - x^k)),
\end{aligned}$$

which matches Equation (3b).

C Theoretical basis for feasibility polishing

For simplicity, we focus on PDHG with a fixed step size less than $1/\|A\|_2$ and a fixed frequency restart scheme. The former choice is typical for analyzing PDHG [26, 27].

Also for simplicity, we assume the LP instance takes the form $\min c^\top x$ such that $Ax = b, x \geq 0$ and focus on primal feasibility polishing (i.e., $c = \mathbf{0}$). However, this analysis generalizes to other settings such as the LP format described in Section 2 and dual feasibility polishing. In this simple case, PDHG reduces to

$$x^{t+1} \leftarrow \underset{x \geq 0}{\mathbf{proj}}(x^t - \eta(c - A^\top y^t)) \tag{13a}$$

$$y^{t+1} \leftarrow y^t - \eta(b - A(2x^{t+1} - x^t)). \tag{13b}$$

Let $X_F := \{x \geq 0 : Ax = b\}$ be the set of feasible solutions. Let $\mathbf{dist}(X, x) := \inf_{z \in X} \|x - z\|_2$. The Hoffman constant of A is the largest constant, $H(A)$, such that

$$H(A) \mathbf{dist}(X_F, x) \leq \|Ax - b\|_2 \quad \text{for all } b \text{ such that } X_F \neq \emptyset. \tag{14}$$

Thanks to Hoffman [55], we know that this constant is always positive.

Lemma 1. Suppose that $c = \mathbf{0}$ (so we have a primal feasibility problem), $0 < \eta < 1/\|A\|_2$ and $y^0 = \mathbf{0}$. Then PDHG, i.e., (13) with $\bar{x}^t = \frac{1}{t} \sum_{i=1}^t x^i$ satisfies

$$\|\bar{x}^t - x^0\|_2 \leq (q + 2) \mathbf{dist}(x^0, X_F) \tag{15}$$

$$H(A) \mathbf{dist}(X_F, \bar{x}^t) \leq \frac{(q + 2) \mathbf{dist}(x^0, X_F)}{\eta t} \tag{16}$$

where $q = 4 \frac{1+\eta\|A\|_2}{1-\eta\|A\|_2}$.

Proof. Let Z^* denote the set of minimax solutions to $\min_{x \geq 0} \max_y b^\top y - y^\top Ax$. From Corollary 2 of [10] we have

$$\|\bar{z}^t - z^0\|_2 \leq (q+2) \mathbf{dist}(z^0, Z^*) = (q+2) \mathbf{dist}(x^0, X_F) \quad (17)$$

which by $\|\bar{x}^t - x^0\|_2 \leq \|\bar{z}^t - z^0\|_2$ gives (15).

Let $\bar{z}^t = \frac{1}{t} \sum_{i=1}^t z^i$. By Corollary 2 of [10] with $C = 1/\eta$ and the fact that $y^0 = \mathbf{0}$ we have

$$\rho_{\|\bar{z}^t - z^0\|_2}(\bar{z}^t) \leq \frac{2\|\bar{z}^t - z^0\|_2}{t\eta}. \quad (18)$$

Moreover, by Applegate et al. [10] equation (22), we have

$$\rho_{\|\bar{z}^t - z^0\|_2}(\bar{z}^t) \geq \|\bar{A}\bar{x}^t - b\|_2 + \|(-c + A^\top \bar{y}^t)^+\|_2 \geq \|\bar{A}\bar{x}^t - b\|_2 \quad (19)$$

where $+$ is the positive part operator. Combining (14), (17), (18), and (19) yields (16). $\square$

Algorithm 5: A first-order method for quickly finding a nearby feasible solution

```

1 Input: A starting point  $x^{0,0}$ , a step-size  $\eta \in (0, \infty)$ , a length for the restart period  $\ell \in \mathbb{N}$ ;
2 Initialize outer loop counter  $n \leftarrow 0$ ;
3 Initialize duals  $y^{0,0} \leftarrow \mathbf{0}$ ;
4 for  $n = 0, 1, 2, \dots$  do
5   initialize the inner loop. inner loop counter  $t \leftarrow 0$ ;
6   repeat
7     Take a step of PDHG, i.e., calculate  $(x^{n,t+1}, y^{n,t+1})$  from  $(x^{n,t}, y^{n,t})$  via (13);
8      $\bar{x}^{n,t+1} \leftarrow \frac{1}{t+1} \sum_{i=1}^{t+1} x^{n,i}$ ;
9      $t \leftarrow t + 1$ ;
10  until  $t \geq \ell$ ;
11  restart the outer loop.  $x^{n+1,0} \leftarrow \bar{x}^{n,t+1}$ ,  $y^{n+1,0} \leftarrow \mathbf{0}$ ,  $n \leftarrow n + 1$ ;
12 end
```

Theorem 1. Suppose that $c = \mathbf{0}$ (so we have a primal feasibility problem). Consider Algorithm 5 with

$0 < \eta < 1/\|A\|_2$ and $\ell \geq t^* := \left\lceil \frac{2(q+2)}{\eta H(A)} \right\rceil$ where $q = 4 \frac{1+\eta\|A\|_2}{1-\eta\|A\|_2}$. Then, for all $n \geq 1$

$$\mathbf{dist}(x^{n,0}, X_F) \leq 2^{-n} \mathbf{dist}(x^{0,0}, X_F) \quad (20a)$$

$$\|x^{n,0} - x^{0,0}\|_2 \leq 2(q+2) \mathbf{dist}(x^{0,0}, X_F). \quad (20b)$$

Proof. We obtain (20a) from (16) and the assumption that $\ell \geq t^*$. Observe that

$$\begin{aligned} \|x^{k,0} - x^{0,0}\|_2 &\leq \|x^{k,0} - x^{k-1,0}\|_2 + \|x^{k-1,0} - x^{0,0}\|_2 \leq (q+2) \mathbf{dist}(x^{k-1,0}, X_F) + \|x^{k-1,0} - x^{0,0}\|_2 \\ &\leq (q+2) 2^{1-k} \mathbf{dist}(x^{0,0}, X_F) + \|x^{k-1,0} - x^{0,0}\|_2 \end{aligned} \quad (21)$$

where the first inequality uses the triangle inequality, the second inequality uses (15), and the last inequality uses (20a). Using (21) will prove the following induction hypothesis:

$$\|x^{n,0} - x^{0,0}\|_2 \leq \left(\sum_{i=1}^n 2^{1-i} \right) (q+2) \mathbf{dist}(x^{0,0}, X_F).$$

In particular, (21) with $k = 1$ shows the hypothesis for $n = 1$ since $x^{k-1,0} = x^{0,0}$. If the hypothesis holds for $n = k-1$ then by (21) it holds for $n = k$. Thus we have proved the induction hypothesis. Using the formula for the sum of an infinite geometric series we get $\sum_{i=1}^n 2^{1-i} \leq \sum_{i=1}^{\infty} 2^{1-i} = 2$, thus the induction hypothesis implies (20b). $\square$

Equation (20a) implies quick convergence of feasibility polishing to a feasible solution. Equation (20b) implies feasibility polishing will converge to a feasible solution that is approximately optimal, assuming the starting point $x^{0,0}$ was approximately optimal. In particular, Equation (20b) implies,

$$|c^\top x^{n,0} - c^\top x^{0,0}| \leq 2(q+2)\|c\|_2 \mathbf{dist}(x^{0,0}, X_F) \leq \frac{2(q+2)\|c\|_2 \|Ax^{0,0} - b\|_2}{H(A)}. \quad (22)$$

Moreover, taking $\eta = \alpha/\|A\|_2$ where $\alpha \in (0, 1)$, as typical [26, 27], gives $q = 4\frac{1+\alpha}{1-\alpha}$. For example, $\alpha = 1/2$ gives $q = 12$. Thus, from Equation (22), we conclude that if we start at a point $x^{0,0}$ with $c^\top x^{0,0} \approx c^\top x^\star$ and $\|Ax^{0,0} - b\|_2 \approx 0$ then

$$\lim_{n \rightarrow \infty} Ax^{n,0} = b \text{ and } \limsup_{n \rightarrow \infty} c^\top x^{n,0} - c^\top x^\star \leq |c^\top x^{0,0} - c^\top x^\star| + \frac{2(q+2)\|c\|_2 \|Ax^{0,0} - b\|_2}{H(A)} \approx 0.$$

D Infeasibility termination criteria

Our infeasibility termination criteria closely follow those of SCS [79] and Applegate et al. [11].

We declare the problem primal infeasible if we find a point $(y, r) \in \mathcal{Y} \times \mathcal{R}$ satisfying:

$$-p(-y; \ell_c, u_c) - p(-r; \ell_v, u_v) > 0, \quad \frac{\|A^\top y + r\|_\infty}{-p(-y; \ell_c, u_c) - p(-r; \ell_v, u_v)} \leq 10^{-9}.$$

Recall that

$$p(y; \ell, u) := u^\top y^+ - \ell^\top y^-,$$

and therefore

$$-p(-y; \ell_c, u_c) - p(-r; \ell_v, u_v) = \ell_c^\top y^+ - u_c^\top y^- + \ell_v^\top r^+ - u_v^\top r^-.$$

We declare the problem dual infeasible if we find a point $(x, s) \in \bar{\mathcal{X}} \times \bar{\mathcal{S}}$ satisfying:

$$c^\top x < 0, \quad \frac{\|Ax + s\|_\infty}{|c^\top x|} \leq 10^{-9}, \quad s \in \bar{\mathcal{S}}, \quad x \in \bar{\mathcal{X}},$$

where $\bar{\mathcal{X}} \subseteq \mathbb{R}^n$ and $\bar{\mathcal{S}} \subseteq \mathbb{R}^m$ are Cartesian products whose i th components are given by

$$\bar{\mathcal{S}}_i := \begin{cases} \{0\} & (\ell_c)_i, (u_c)_i \in \mathbb{R}, \\ \mathbb{R}^+ & (\ell_c)_i = -\infty, (u_c)_i \in \mathbb{R}, \\ \mathbb{R}^- & (\ell_c)_i \in \mathbb{R}, (u_c)_i = \infty, \\ \mathbb{R} & \text{otherwise;} \end{cases} \quad \text{and} \quad \bar{\mathcal{X}}_i := \begin{cases} \{0\} & (\ell_v)_i, (u_v)_i \in \mathbb{R}, \\ \mathbb{R}^+ & (\ell_v)_i \in \mathbb{R}, (u_v)_i = \infty, \\ \mathbb{R}^- & (\ell_v)_i = -\infty, (u_v)_i \in \mathbb{R}, \\ \mathbb{R} & \text{otherwise.} \end{cases}$$

Remarks:

- The threshold of 10^{-9} was selected to ensure that none of the large-scale test instances would incorrectly terminate with an infeasibility declaration.
- All instances were rescaled prior to solving (Section 6), which implicitly affects the infeasibility termination criteria in a manner analogous to how rescaling affects the optimality termination criteria; see Equation (10).

E Detailed statistics for PDLP on the large-scale instances

Table 5: Detailed statistics for PDLP on the large-scale instances. KKT passes refers to the total number of times that Ax and $A^T y$ are computed. Residuals are measured in ℓ_∞ -norm, i.e., absolute error (on the instances scaled as per Section 6). The gap is the difference between the primal and dual objective. Rows with gray background indicate unsuccessful termination (i.e., either hitting the six day time limit or in the case of qap-tho-150, a numerical error as per Table 3), in which case we report the statistics from the most recent recorded iteration. Memory is the maximum memory usage in GB according to SLURM. Note that finer grained information is available at <http://www.oliverhinder.com/large-scale-lp-problems>.

Instance name	Time (hrs)	KKT passes	Objective			Residuals		Memory (GB)
			Primal	Dual	Gap	Primal	Dual	
With polishing								
design-match	9.34	1.5E+4	2.87E+6	2.87E+6	5.5E-2	4.0E-11	0	135
heat-source-easy	59.97	1.9E+6	5.16E+7	5.16E+7	9.1E-4	9.9E-9	9.3E-9	18
heat-source-hard	—	3.8E+6	4.67E+7	4.69E+7	-1.6E+5	6.2E+0	9.7E+2	17
mediterranean-shipping	32.62	1.1E+5	-1.07E+3	-1.07E+3	6.9E+0	7.8E-9	0	81
production-inventory	—	1.6E+6	4.50E+4	4.50E+4	3.3E-2	2.7E-7	5.9E-2	29
qap-tho-150	0.14	2.4E+2	0	-4.23E+35	4.2E+35	1.0E+0	0	96
qap-wil-100	0.28	3.8E+3	2.21E+5	2.21E+5	1.1E+2	5.7E-9	0	22
supply-chain	18.90	6.1E+4	1.08E+8	1.08E+8	6.7E+3	5.6E-9	0	70
tsp-gaia-100m	21.06	7.8E+3	1.94E+9	1.94E+9	5.8E+4	7.6E-9	0	611
tsp-gaia-10m	2.99	7.9E+3	1.30E+8	1.30E+8	3.0E+3	6.6E-9	0	40
world-shipping	53.81	2.2E+5	-1.18E+5	-1.18E+5	3.7E+2	9.9E-9	0	91
Without polishing								
design-match	67.95	1.1E+5	2.87E+6	2.87E+6	-8.7E-6	9.8E-9	0	136
heat-source-easy	—	3.9E+6	5.16E+7	5.16E+7	-2.3E-2	2.8E-6	4.7E-5	15
heat-source-hard	—	3.5E+6	4.70E+7	4.76E+7	-6.6E+5	1.2E+1	7.2E+2	15
mediterranean-shipping	—	5.5E+5	-1.07E+3	-1.07E+3	6.7E+0	7.4E-6	0	72
production-inventory	—	1.3E+6	4.50E+4	4.50E+4	8.7E-2	2.9E-7	5.9E-2	27
qap-tho-150	0.14	2.4E+2	0	-4.23E+35	4.2E+35	1.0E+0	0	97
qap-wil-100	44.35	6.4E+5	2.21E+5	2.21E+5	1.6E-4	6.6E-9	1.0E-8	20
supply-chain	—	8.5E+5	1.08E+8	1.08E+8	5.9E+1	2.8E-3	1.9E-5	60
tsp-gaia-100m	—	5.6E+4	1.94E+9	1.94E+9	2.8E+1	1.8E-3	0	552
tsp-gaia-10m	—	3.9E+5	1.30E+8	1.30E+8	2.3E-1	3.6E-5	0	37
world-shipping	—	2.4E+5	-1.18E+5	-1.18E+5	7.4E+1	8.5E-2	0	80

F Additional numerical experiments

To supplement the experiments on large-scale linear programs in the body of the paper, we ran some additional experiments to verify the performance on the Mittelman benchmark (Section F.1), a standard collection of slightly smaller instances, and on a set of infeasible instances (Section F.2). For consistency, PDLP was set up exactly the same as it was for the large-scale experiments, including keeping presolved disabled, rescaling the instances, and maintaining the same termination criteria. For these experiments, we used Gurobi version 12.0.3 instead of 11.0.2 (the version used for the large-scale experiments in Section 7) because at the time of running these experiments, the Pitt computing cluster we use had updated its Gurobi version.

F.1 Numerical results on Mittelman’s benchmark instances

We compared PDLP with and without feasibility polishing on the Mittelman test set. Our results are reported in Figure 3, which shows that feasibility polishing is never much slower than standard PDLP and can give orders of magnitude speed improvements, consistent with our large-scale numerical results in Section 7.

We also tested the impact of reducing the feasibility tolerance (both for the primal and dual). The default feasibility tolerance is 10^{-8} as specified in Equation (9). These results are reported in Figure 4, which shows that PDLP with feasibility polishing is insensitive to reductions in feasibility tolerance, while PDLP without feasibility polishing suffers significant performance degradation as the feasibility tolerance is reduced. Without polishing, one additional instance failed to solve at 10^{-10} feasibility tolerance, and a further eight additional instances failed at 10^{-12} feasibility tolerance. With polishing, one additional instance failed to solve at 10^{-10} feasibility tolerance, and a further four additional instances failed at 10^{-12} feasibility tolerance.

Table 7 reports the results for PDLP (with and without polishing), and Gurobi (barrier, primal simplex, and dual simplex) on the Mittelman benchmark instances. From this table it is clear that, with the exception of three instances, Gurobi is almost always faster. However, it is important to keep in mind that this CPU implementation of PDLP studied in this paper is not intended for tackling this scale of problem and a GPU implementation is likely more suitable for such a comparison.

F.1.1 Instances incorrectly declared infeasible

PDLP, both with and without polishing, fails to find an optimal solution in 14 of 49 instances. Two of these failures occur because the instance was declared infeasible by PDLP (both with and without feasibility polishing): pds-100 and a2864. We found that dropping the infeasibility termination tolerance to 10^{-10} prevents pds-100 from being declared infeasible, allowing the problem to be solved in 1075.1 seconds with feasibility polishing and in 1727.3 seconds without feasibility polishing. Decreasing the infeasibility termination tolerance for a2864 does not prevent it from being declared infeasible, and detailed investigation indicates that the issue is likely a small numerical error either in the computation of the infeasibility criteria or in the rescaling scheme (See <https://github.com/google/or-tools/issues/4874>). Manually modifying the infeasibility criteria from $-p(-y; \ell_c, u_c) - p(-r; \ell_v, u_v) > 0$ to $-p(-y; \ell_c, u_c) - p(-r; \ell_v, u_v) > 10^{-14}$ fixes the issue, allowing the problem to be solved in 116.4 seconds with feasibility polishing and in 351.9 seconds without feasibility polishing.

Table 6: Statistics for Mittelman’s linear programming collection. Instances collected are based on the benchmark pages from 2025-06-18, which can be retrieved from archive.org: <https://web.archive.org/web/20250624111618/https://plato.asu.edu/ftp/lpfeas.html>.

problem	# constraints	# variables	# nonzeros
L1_sixm250obs	986069	428032	4280320
Linf_520c	93326	69004	566193
a2864	22117	200787	20078717
bdry2	376500	250998	1500003
cont1	160793	40398	399991
cont11	160793	80396	439989
datt256	11077	262144	1503732
dlr1	1735470	9121907	18365107
ex10	69609	17680	1179680
flnw-bin1	772872	1141653	8611326
fome13	48569	97840	334984
graph40-40	360900	102600	1260900
irish-e	104260	61728	538809

Continued on next page

Table 6 – *Continued from previous page*

problem	# constraints	# variables	# nonzeros
neos	479120	36786	1084461
neos3	512209	6624	1542816
neos-3025225	91572	69846	9357951
neos5052403	38269	32868	4898304
neos-5251015	486531	136971	1955388
ns1687037	50622	43749	1406739
ns1688926	32768	16587	1712128
nug08-3rd	19728	20448	139008
pds-100	156244	505360	1390539
psched3-3	266228	79555	1062480
qap15	6331	22275	110700
rail02	95791	270869	756228
rail4284	4284	1092610	12372358
rmine15	358395	42438	879732
s82	87878	1690631	7022608
s100	14734	364417	2127672
s250r10	10963	273142	1572104
savsched1	295990	328575	1846351
scpm1	5000	500000	6250000
shs1023	133944	444625	1044725
square41	40161	62234	13628623
stat96v2	29089	957432	2852184
stormG2_1000	528186	1259121	4228817
stp3d	159488	204880	662128
support10	165685	14770	551152
tpl-tub-ws16	1154615	747691	4720567
woodlands09	194599	382147	2646003
Dual2_5000	30000600	33050602	93001800
Primal2_1000	1299380	2559380	5498140
thk_48	6366377	8609262	27802878
thk_63	5694387	7701112	21592414
L1_sixm1000obs	3082940	1426256	14262560
L2CTA3D	210000	10000000	30000000
degme	185501	659415	8127528
dlr2	7132926	38868107	78091589
set-cover	10000	1102008	20442268

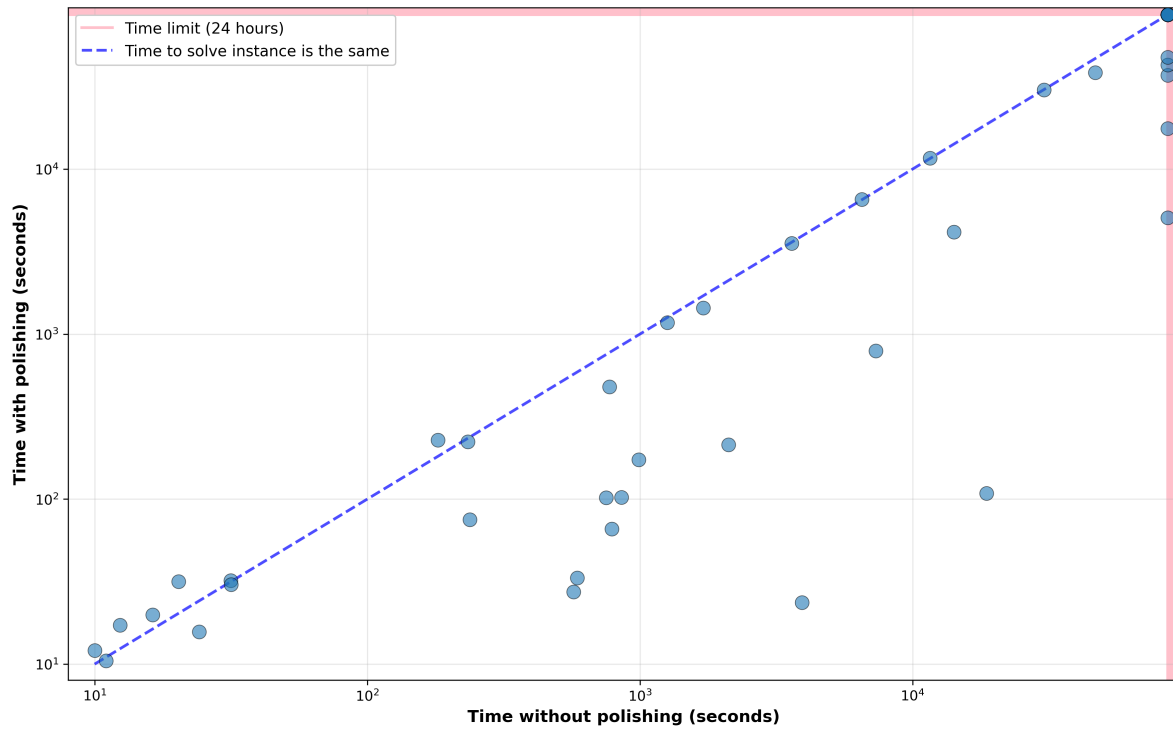


Figure 3: Comparison of PDL with and without feasibility polishing for the Mittelman test set. This test set consists of 49 problems, 14 of which were unsolved by both with and without polishing. Note that all problems solved without polishing were also solved with the polishing option enabled.

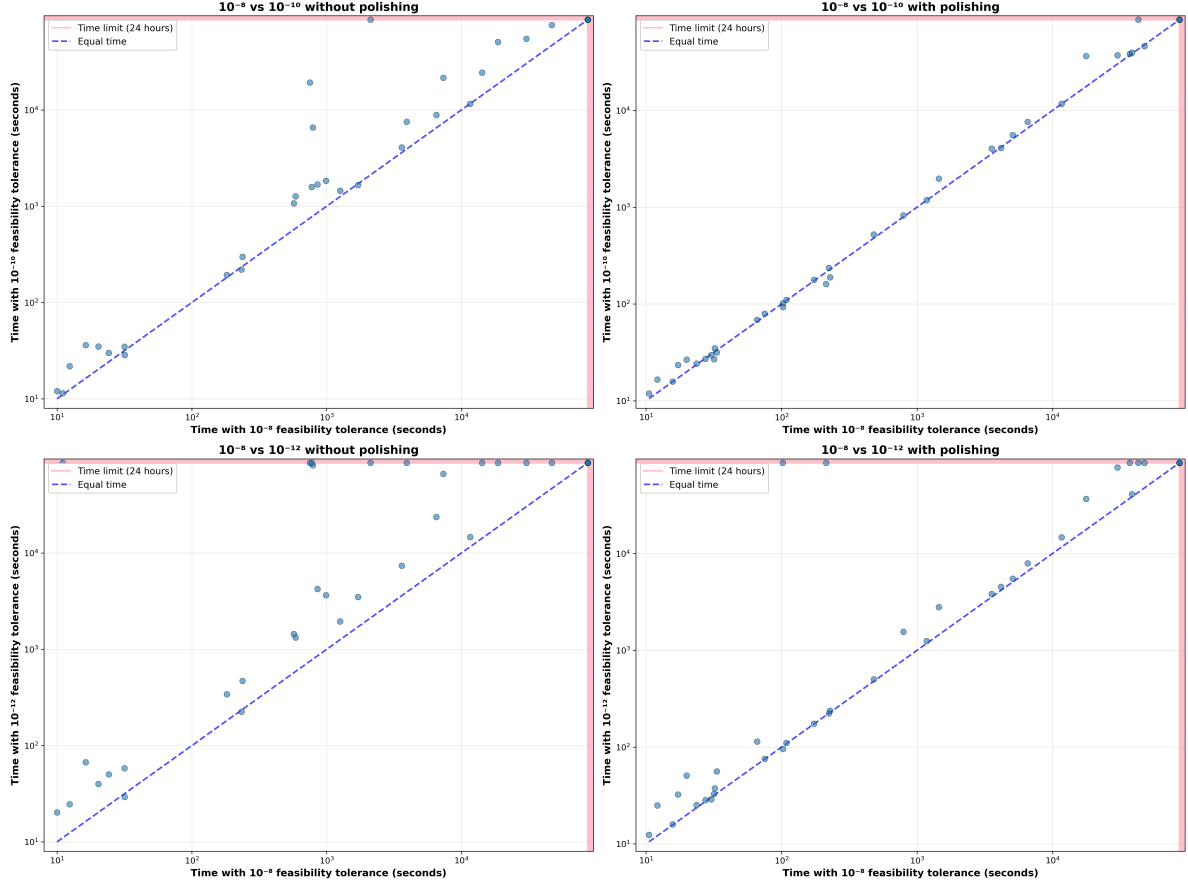


Figure 4: Comparison of PDLP with and without feasibility polishing for the Mittelman test set with different feasibility tolerances (both for the primal and dual). The standard feasibility tolerance in this paper is 10^{-8} .

Table 7: Computational results for Mittelman benchmark collection. The value — signifies the problem failed to solve in the 24 hour time limit, † indicates the method ran out of memory, and × signifies the problem was incorrectly declared infeasible (See discussion in Section F.1.1). One KKT pass refers to the calculation Ax and $A^T y$.

Problem	PDLP				Gurobi (minutes)		
	KKT Passes		Time (minutes)		Barrier	Simplex	
	no polish	polish	no polish	polish		Primal	Dual
Dual2_5000	79,497	79,497	193.36	194.59	†	—	59.19
L1_sixm1000obs	69,577	7,049	35.19	3.55	1.46	2.18	0.53
L1_sixm250obs	53,324	6,733	12.53	1.70	0.11	0.11	0.09
L2CTA3D	19,872	987	13.16	1.10	22.03	—	—
Linf_520c	—	—	—	—	0.09	9.20	1.00
Primal2_1000	—	922,873	—	293.87	—	1181.92	1236.06
a2864	×	×	×	×	0.03	5.06	6.20
bdry2	—	—	—	—	0.27	71.18	96.51
cont1	—	3,331,570	—	618.30	0.03	5.13	5.89
cont11	—	3,693,319	—	714.41	0.04	38.09	25.66
datt256	1,347	1,499	0.27	0.33	0.03	225.96	5.28
degme	—	—	—	—	0.73	—	—
dlr1	—	—	—	—	0.91	210.87	39.98
dlr2	—	—	—	—	9.14	—	—
ex10	710	788	0.17	0.20	—	0.06	0.08
fhnw-bin1	357,434	357,594	108.48	109.10	2.54	0.45	3.86
fome13	—	440,673	—	84.78	0.02	0.12	0.13
graph40-40	1,090	1,286	0.21	0.29	0.02	0.39	0.12
irish-e	—	—	—	—	0.05	0.74	0.39
neos	—	4,054,159	—	794.93	0.15	1.87	0.11
neos-3025225	11,141	11,141	3.02	3.80	0.21	7.83	2.55
neos-5251015	2,805	2,805	0.53	0.54	0.06	47.87	0.05
neos3	854	822	0.18	0.17	0.03	0.43	0.05
neos5052403	40,302	1,700	9.51	0.46	0.08	2.83	0.85
ns1687037	—	—	—	—	0.08	10.56	4.58
ns1688926	—	—	—	—	0.03	1.16	0.02
nug08-3rd	1,537	1,689	0.34	0.53	0.01	0.82	0.62
pds-100	×	×	×	×	0.25	1.97	0.17
psched3-3	—	—	—	—	0.13	1.51	1.74
qap15	50,434	1,926	9.81	0.55	0.01	0.21	0.30
rail02	—	—	—	—	0.69	52.74	39.41
rail4284	219,460	960	65.48	0.39	0.32	30.54	0.80
rmine15	70,824	7,834	14.26	1.70	0.25	10.29	8.79
s100	117,222	105,378	20.99	19.60	0.12	1.33	1.71
s250r10	2,812,856	2,864,646	505.42	502.39	0.11	1.41	0.32
s82	2,082,799	1,808,861	780.22	642.19	1.26	165.40	20.05
savshed1	1,955	966	0.40	0.26	0.13	0.46	1.74
scpm1	1,815	1,639	0.53	0.51	0.25	13.33	4.54
set-cover	688,191	3,385	310.96	1.80	1.69	755.10	1041.93
shs1023	—	—	—	—	0.29	17.77	1.12
square41	242,433	232,357	60.14	59.14	0.03	0.06	0.05
stat96v2	—	—	—	—	0.38	108.24	30.46
stormG2_1000	48,713	30,365	12.90	7.98	0.58	13.71	0.38

Continued on next page

Table 7 – *Continued from previous page*

Problem	PDLP				Time for Gurobi		
	KKT Passes		Time		Barrier	Simplex	
	no polish	polish	no polish	polish		Primal	Dual
stp3d	92,890	15,187	16.48	2.89	0.07	10.50	1.39
support10	21,540	6,736	3.96	1.25	0.06	0.69	0.28
thk_48	140,512	14,658	122.21	13.20	58.65	—	—
thk_63	32,774	29,894	28.45	24.06	2.18	—	12.94
tpl-tub-ws16	713,992	208,841	236.37	69.26	0.54	6.00	13.90
woodlands09	17,870	17,870	3.89	3.72	0.08	4.77	8.32

F.2 Numerical results for primal infeasibility detection

We compared PDLP with and without feasibility polishing on the NETLIB infeasibility test set (See <https://www.netlib.org/lp/infeas/readme>). This test set mostly comprises smaller instances (See Table 8), but it can be a good test of robustness.

PDLP performance on this set is reported in Table 9. Overall, PDLP’s performance on these problems is less than desired, with it only solving 15/29 problems. Thus, improving the ability of first-order methods to reliably detect infeasibility is an important subject for future research. We also find that on these instances, feasibility polishing has no notable impact on solver performance.

We also tested Gurobi on these problems but do not report the results because it was so dominant. In particular, every instance was declared primal infeasible by at least one Gurobi method in less than 0.1 seconds, except ‘pang’, which was declared either infeasible or unbounded by both dual and primal simplex in 0.01 seconds.

Table 8: Problem statistics for primal infeasible linear programs.

problem	# constraints	# variables	# nonzeros
bgdbg1	348	407	1440
bgetam	400	688	2409
bgindy	2671	10116	65502
bgprtr	20	34	64
box1	231	261	651
ceria3d	3576	824	17602
chemcom	288	720	1566
cplex1	3005	3221	8944
cplex2	224	221	1058
ex72a	197	215	467
ex73a	193	211	457
forest6	66	95	210
galenet	8	8	16
gosh	3792	10733	97231
gran	2658	2520	20106
greenbea	2393	5405	30883
itest2	9	4	17
itest6	11	8	20
klein1	54	54	696
klein2	477	54	4585
klein3	994	88	12107
mondou2	312	604	1208
pang	361	460	2652
pilot4i	410	1000	5141
qual	323	464	1646
reactor	318	637	2420
refinery	323	464	1626
vol1	323	464	1646
woodinfe	35	89	140

Table 9: Computational results for the collection of primal infeasible linear programs given in Table 8. The time limit for these problems was one hour. One KKT pass refers to the calculation Ax and $A^\top y$.

Problem	Time (seconds)		KKT Passes	
	no polishing	polishing	no polishing	polishing
bgdbg1	104.0	87.6	7379	7379
bgetam	239.9	283.7	22160	22160
bgindy	1.9	2.0	68	68
bgrptr	14.3	15.6	1857	1857
box1	139.0	142.0	11973	11973
ceria3d	—	—	—	—
chemcom	34.7	36.2	3081	3081
cplex1	—	—	—	—
cplex2	—	—	—	—
ex72a	360.6	380.7	31990	31990
ex73a	438.1	554.3	41611	41623
forest6	64.3	66.6	5570	5570
galenet	0.1	0.1	67	67
gosh	—	—	—	—
gran	—	—	—	—
greenbea	231.4	214.5	18131	18131
itest2	0.1	0.1	206	206
itest6	0.1	0.1	66	66
klein1	—	—	—	—
klein2	—	—	—	—
klein3	—	—	—	—
mondou2	113.7	114.2	7693	7693
pang	—	—	—	—
pilot4i	—	—	—	—
qual	—	—	—	—
reactor	—	—	—	—
refinery	—	—	—	—
vol1	—	—	—	—
woodinfe	41.4	37.8	3969	3969